

Educational Security Animations

Edward James Daft

BSc Computer Science with Security & Forensics

School of Computer Science and Informatics, Cardiff University

Supervisor: George Theodorakopoulos

Moderator: Crispin Cooper

4th May 2026

Abstract

Cryptography plays a critical role in modern computing, but many of its core concepts can be difficult for learners to understand due to their complexity. In particular, modes of operation such as Counter (CTR) and Galois/Counter Mode (GCM) require an understanding of how plaintext is transformed over a multitude of steps, which may not be effectively conveyed through traditional teaching methods.

This project aims to develop an educational application that can help to improve understanding of these key concepts through interactive and visual animations. This was achieved through modelling these concepts, using simplified implementations that are appropriate for undergraduate level students.

Lessons about these algorithms are delivered through step-by-step animation sequences, followed by quizzes and supported by features such as a glossary, progress tracking and accessibility focused features. These animations were implemented using a custom animation framework built to overcome limitations caused by the selected user interface library. These different elements of the system were implemented using a modular codebase structure, allowing for extensions to the system.

The outcome of this project demonstrates that visual and interactive experiences can provide a clearer understanding when it comes to cryptographic concepts, particularly for complex examples like CTR and GCM.

Table of Contents

1. Introduction	5
2. Background	6
2.1 Context.....	6
2.2 Problem Definition	6
2.3 Stakeholders	7
2.4 Existing solutions.....	7
2.5 Theoretical Background.....	8
2.6 Research Questions	9
3. Specification	10
3.1 System Overview	10
3.2 Functional Requirements	10
3.3 Non-Functional Requirements.....	12
3.4 Success Criteria	13
4. Design	14
4.1 Design Overview	14
4.2 System Architecture	14
4.3 User Interface & Navigation	15
4.4 User Interaction	20
4.5 Lesson Design.....	21
4.6 Data Flow & Storage	24
4.7 Design Constraints	24
5. Implementation.....	26
5.1 Implementation Overview.....	26
5.2 User Interface.....	26
5.3 System Structure.....	28
5.4 Key Components.....	30
5.5 Challenges	36
5.6 Debugging Methodology	38
6. Results & Evaluation	39

6.1 Testing Approach	39
6.2 Functional Requirements Evaluation	39
6.3 Non-Functional Requirements Evaluation	41
6.4 Usability Evaluation	42
6.5 Success Criteria Evaluation	43
6.6 Further Testing & Evaluation	44
7. Conclusion & Future Work.....	46
7.1 Conclusion	46
7.2 Future Work	47
8. Reflection	48
List of Abbreviations	49
Glossary	49
References.....	51
Appendix	52

Introduction

Understanding how modern cryptographic systems operate is an important aspect of computer science, particularly due to their importance in everyday digital interactions and communications. However, many of the underlying concepts can be difficult for learners to fully grasp using traditional teaching methods alone.

The aim of this project is to develop an educational system that improves understanding of cryptographic processes through interactive and visual representations. The project focuses on block cipher modes of operation, specifically Counter (CTR) and Galois/Counter Mode (GCM), which are commonly used in modern encryption systems.

The system is intended for learners with a foundational understanding of computing, such as undergraduate level students. These users may benefit from a clearer, step-by-step view of how encryption algorithms transform data. These views will be conceptual and educational, using simplified implementations to prioritise clarity over full detail.

To achieve this the project adopts a visual approach to explaining cryptographic algorithms, allowing users to observe how data is processed at each stage. This includes illustrating keystream generation, XOR operations, and how additional inputs such as initialisation vectors contribute to security.

The outcome of the project is an educational tool that aims to support improved understanding of cryptographic modes of operation, specifically GCM and CTR, and an evaluation of its successfulness through a range of testing and heuristic evaluations.

Background

2.1 Context

Cryptography is a core topic within modern computer science education, particularly at undergraduate level, due to its role in protecting systems we use every day (Stallings, 2017). As applications such as messaging platforms have become increasingly prevalent, an understanding of how cryptographic algorithms function is essential for computer science students.

Despite this, the teaching of these concepts often relies on static diagrams and written explanations. While these materials are effective at explaining the high-level concepts, they can be limited in how they demonstrate how the algorithms operate in practice. This is especially true for more complex processes, where understanding the algorithm requires following multiple sequential transformations of data.

As a result, visual and step-based representations can provide learners with a clearer understanding of how data is changed by these algorithms, helping to bridge the gap between high level understanding of what the system does, and lower-level understanding of how it achieves it.

2.2 Problem Definition

A key issue that students may struggle with is following how these cryptographic functions operate in practice. This is particularly evident when studying more complex topics such as modes of operation, where multiple sequential steps must be followed to transform data. While the overall purpose of these algorithms may be understood through the use of static diagrams and explanations, the detailed steps of their function are often hard to visualise and fully comprehend.

This difficulty is largely caused by the way these concepts are presented to learners. Static diagrams and written explanations, whilst adequate for simple levels of understanding, may fail to effectively teach how the data changes at each stage of the operation, and how these changes compound to create a secure algorithm. As a result of this, students may be able to understand the overall theory behind an algorithm, without understanding how it functions step-by-step.

Therefore, there is a need for a system that presents cryptographic functions in a more interactive and structured way, that allows users to observe each stage of an algorithm, while progressing at their own pace. This would help to improve understanding and engagement with these concepts, as research has shown (Chernikova et al., 2020).

2.3 Stakeholders

The key stakeholders for this project are students studying cryptography at an undergraduate level, particularly those that are new to the subject and may struggle to understand complex cryptographic algorithms. Students need a solution that provides clear explanations that allow them to learn the content that is presented to them, as well as opportunities to test and reinforce their understanding. At this educational level, students do not require a full implementation-level understanding of the algorithms, instead, they need to grasp the underlying concepts and processes. Therefore a degree of simplification is acceptable, provided that the core principles and overall accuracy of the algorithms are preserved.

Secondary stakeholders include educators, who may use the system as a supplementary teaching tool alongside their own material. For this group of stakeholders, accuracy and clarity is particularly important, as the system must align with established descriptions of cryptographic algorithms so as not to contradict with teaching materials.

The system may also be used by independent learners outside of formal education, who require a self-contained learning resource. For those users, as well as students, supporting features such as a glossary are highly important to provide additional context which supports independent learning.

2.4 Existing Solutions

Whilst some similar solutions to this cryptography education problem exist, they all have flaws that make them unsuitable for the goal of teaching modes of operation.

The most common existing solution to this problem is already existing teaching materials. These are usually static explanations that rely on diagrams and are often text heavy. While these materials are common, students may struggle to gain a deeper understanding of the underlying processes without being able to visualise the algorithm.

One of the most similar, and established, educational platforms is CrypTool (CrypTool Portal, n.d.), which is a desktop and web-based platform for exploring cryptographic algorithms. It allows users to construct their own cryptographic algorithms and experiment with existing ones. This tool is highly broad in its scope, covering modes of operation to an extent. However, whilst CrypTool does have support for modes of operation, such as CTR and GCM, they are simply presented as a tool to experiment with. Key processes within these algorithms such as keystream generation and authentication tag construction are abstracted from the user. This reduces the effectiveness of this tool for students wanting to learn these specific concepts. This solution prioritises coverage of topics rather than depth for any given algorithm, whereas the proposed solution will cover a narrower scope but allow for a much greater depth.

In addition to these educational tools, there are a range of libraries that offer cryptographic functions such as the Crypto++ library for C++. These provide

implementations of a wide range of modes including GCM and CTR. However, these libraries are focused on real-world practical applications of these algorithms rather than educational value. They abstract the internal process from the user, meaning that they are unsuitable for teaching purposes, as learners cannot interact with and visualise the algorithm.

These existing solutions demonstrate that there is a lack of systems that allow step-by-step visualisations of these specific modes of operation. Other solutions usually do not support GCM and CTR, or do so at a highly abstracted level, meaning that they are not an effective learning tool for a student that wants to gain a deeper understanding of how these algorithms function.

2.5 Theoretical Background

Cryptography is a fundamental component of secure computing systems, designed to protect information as it is stored and transmitted. Its primary aim is to ensure confidentiality, integrity, and authentication of data transmitted (Stallings, W, 2017, Ch.1). These properties form the foundation of secure digital communication and are essential in preventing a wide range of security threats.

In practical use cases, these goals are not all achieved through a single technique, but rather through a combination of multiple cryptographic primitives working together. Basic encryption provides confidentiality, while additional mechanisms such as authentication tags ensure integrity and authenticity. This layered approach demonstrates the complexity of real-world security applications. Modern systems such as messaging applications and web communication protocols such as HTTPS, demonstrate this concept of combining techniques.

Key cryptographic primitives such as the XOR (Dworkin,M, 2001, Ch. 2), exclusive or, operation are commonly used within encryption algorithms due to its reversibility and speed of calculation. This results in the operation commonly being used in stream ciphers as it allows for reversible masking of data using a keystream. A stream cipher is a type of encryption that processes data one bit at a time, allowing for efficient encryption of data with varying length.

A block cipher is a cryptographic algorithm that operates on fixed-size chunks of data, transforming a block of plaintext into a corresponding block of ciphertext using a secret key. Because the input size is strict and limited, block ciphers on their own are not suitable for encrypting messages of arbitrary length. Therefore, modes of operation define how block ciphers can be repeatedly applied in order to encrypt amounts of data larger than a block. Modes of operation also usually contain additional mechanisms such as counters or randomisation that ensure blocks are encrypted in an unpredictable way. Meaning that two identical plaintext blocks would have different corresponding ciphertext.

CTR (Counter) mode of operation (Dworkin,M, 2001) is one such example of these algorithms. It converts a block cipher into a stream-like cipher by encrypting a counter value to create a keystream, which is then XOR with the plaintext block. The

counter value makes use of an initialisation vector (also called a nonce) which ensures that each block is unique. The key features of CTR include the fact that it is parallelisable and encryption and decryption are identical operations.

GCM, Galois/Counter Mode (Dworkin, M, 2001), is built directly on CTR mode. The encryption process is the same, but it adds an authentication layer on top of the encryption. This is achieved through calculating an authentication tag, which is generated using Galois field multiplication. Providing authentication prevents tampering and ensures the integrity of data sent.

These principles and algorithms form the theoretical foundation for this project, which focuses on modelling and visualising these encryption modes in an educational context to support understanding of how secure communication systems operate.

2.6 Research Questions

Throughout the process of developing and documenting this project, the following research questions will be considered and addressed:

1. How can fundamental principles of cryptography, such as XOR and block cipher modes, be effectively represented in a simple educational tool?
2. How does CTR mode operate and encrypt data, and how can this process be visualised to improve understanding?
3. How does Galois/Counter Mode extend the functionality of CTR to include authentication, and what is the impact of this layer?
4. To what extent can simplified cryptographic algorithms accurately reflect the behaviour of real-world systems, while still remaining accessible for an educational context?

Specification

3.1 System Overview:

The system is an educational tool designed for computer science students studying cryptography, with a focus on modes of operation. It provides visual representations of complex cryptographic algorithms through short, step-by-step animations. Allowing students to develop an understanding of how these algorithms function, in a structured and accessible manner.

The system is interactive, allowing users to control the progression of each animation, and engage with the material at their own pace. To reinforce learning, each lesson is followed by a short quiz, which assesses the user's understanding of the concepts presented.

In addition, the system includes supporting features such as a glossary and progress tracking which are intended to improve the learning experience. Accessibility features such as alternative colour schemes and text size adjustability are also included to support the user.

The system is implemented as a desktop application, and is designed to run on any average hardware, running Windows 11, in order to ensure the tool is accessible to as many students as possible.

3.2 Functional Requirements:

FR1: Animation of cryptographic algorithms – The system shall show step-by-step animations that demonstrate the function of cryptographic algorithms.

This ensures that the user can observe how cryptographic algorithms function in a way that is engaging and understandable.

FR2: Supported algorithms – The system shall have animations covering the topic of modes of operation, specifically, GCM encryption & decryption, CTR encryption & decryption.

This ensures the system focuses on challenging modes of operation that are often difficult for students to learn and understand.

FR3: Quiz functionality – The system shall have small quizzes after every animation that test the user's knowledge of the concept presented. These quizzes shall give feedback to the user once submitted.

This requirement ensures that the user has understood the content that has been presented, rather than skipping through the animation, therefore not engaging with the lesson's concepts.

FR4: Animation control – The system shall allow the user to move forwards and backwards through animation steps.

This requirement allows the user to navigate through the content at their own pace to ensure that they understand it. If they are confident on a step, they can move to the next, if they need it explained again then they can revisit the previous step to replay it.

FR5: Theme selection – The system shall allow the user to select between light and dark modes.

This requirement ensures that the system is accessible to as many students as possible. Light/dark modes cater to those with light sensitivity and other conditions as well as those with a preference for darker interfaces (W3C, 2025).

FR6: Text size selection – The system shall allow users to change the text size.

This requirement also improves accessibility of the system, particularly for those who have visual impairments and may struggle to read smaller text. It also allows users to modify their experience to suit their preferences, which increases accessibility (W3C, 2025).

FR7: Glossary – The system shall include a glossary.

This requirement supports the user's learning by providing access to definitions of technical terms, which are common in cryptography, and so will appear frequently in lessons. Without a glossary, users may have to research the term which would interrupt the learning experience.

FR8: Progress tracking – The system shall track the user's progress, including after the program has been closed. Progress should be displayed to the user.

This requirement allows the user to learn over multiple sessions, allowing them to engage with the material at their own pace. It encourages engagement by providing a clear indication of progress. Visible progress can encourage continued use of the system.

FR9: Navigation – The system shall allow users to navigate between different sections of the application.

This requirement ensures that users can navigate to and access all sections of the system to allow them to take full advantage of the resources provided by it.

3.3 Non-Functional Requirements:

NR1. Usability – The system shall provide an intuitive user interface.

This ensures that the system can be used without requiring training of any kind, increasing accessibility.

NR2. Accessibility – The system shall be accessible for users with varying visual needs through customisable text size and display themes.

This ensures that the system is readable and comfortable to use for a wide range of users, including those who may have sensitivity to light and visual impairments.

NR3. Performance – The system shall respond to user input without any noticeable delay.

This requirement ensures that the system feels responsive which provides a smooth and uninterrupted learning experience.

NR4. Reliability – The system shall operate reliably, and as expected, during normal use, and should not experience any unexpected crashes or loss of functionality.

Crashes can be distracting for the user, and loss of functionality prevents the user from continuing their learning as expected. This requirement ensures that neither of these issues arise under normal conditions, ensuring an uninterrupted learning experience for the user.

NR5. Compatibility – The system shall operate on any standard desktop running Windows 11 without requiring specialised hardware. It shall also allow for full functionality without any internet access.

This requirement ensures that the system is functional on the standard hardware that students are used to interacting with. It also ensures that the system remains functional without internet access, something which was identified as an issue in similar systems (see Section 2.4), allowing users to learn from anywhere.

NR6. Scalability - The system shall be designed to be scalable, allowing new lessons to be created and added to the system without significant restructuring of the program.

The current system only supports a small number of the many cryptographic algorithms. Supporting scalability in the design of the system allows additional

lessons to be implemented without significant challenges. Therefore ensuring that the system can evolve over time.

3.4 Success Criteria:

The system will be considered successful if the following criteria are met:

SC1. Feature functionality – All core features, animations, quizzes, glossary, progress tracking, settings, and navigation between features, function as defined (see 3.2) and pass functional testing.

SC2. Accuracy of lesson content – All lesson content is correct and aligned with established descriptions of the algorithms. If simplifications have been made in order to make concepts easier to understand, they must be explained and justified to the user.

SC3. Maintainability and Extensibility – The system is modular, allowing new lessons and features to be added without modifying core components of the application.

SC4. User interaction – All user interactions behave consistently across the system, widgets, such as buttons, look uniform and perform actions that align with their labelled function.

SC5. Learning effectiveness – The system supports user understanding of cryptographic concepts through clear, step-based explanations and interactive elements.

Design

4.1 Design Overview

The system has been designed to realise the requirements outlined in Section 3. This is achieved through a structured and modular system. It is designed as an interactive, graphical user interface based educational tool, allowing users to engage with cryptographic processes through visualisations.

The system adopts a modular architecture where functionality is split logically which allows each component to operate independently, whilst still contributing to the system as a whole. A step-based animation system is used for the visualisations which gives greater user control and allows complex algorithms to be decomposed into individual steps of the process.

4.2 System Architecture

Figure 4.2.1 shows the high-level system architecture. The design has a layered structure, separating the user interaction, application logic and systems such as animations and quizzes.

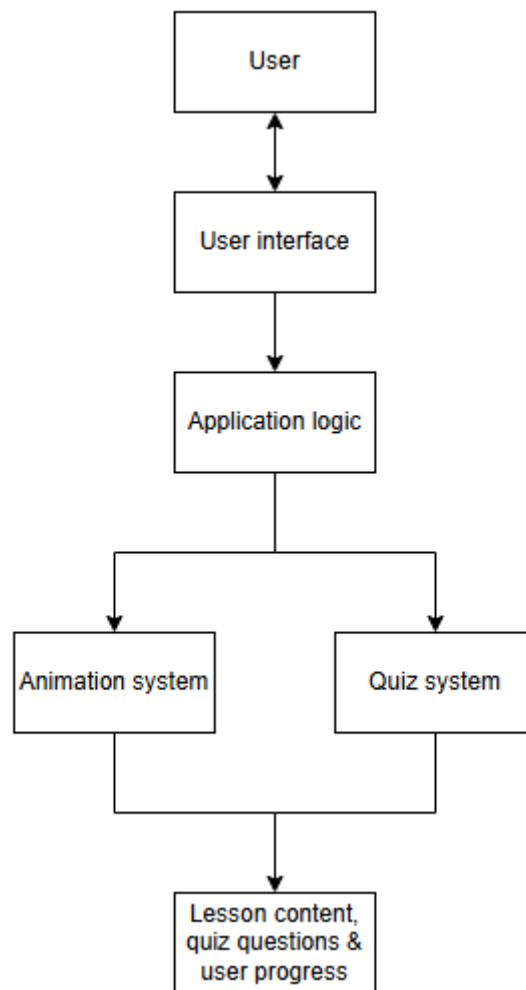


Figure 4.2.1 System architecture diagram

The user interacts with the system through the interface, which communicates with the application logic to handle user input and navigation, and then the user interface changes to reflect these actions.

The animation system is what is responsible for delivering the step-by-step visualisations, while the quiz system handles assessment after the content has been delivered. Both of these components access, and in some cases modify, persistent data storage files, such as user progress information and settings preferences.

The separation of components improves maintainability by isolating functions and allowing them to be worked on individually, without impacting the wider system. Scalability is supported, as adding new lessons does not require changing the architecture of the whole system.

Using a layered architecture ensures separation of concerns, making sure the user interface, application logic, and functional components operate independently. The application logic acts as a central controller, managing how the different components interact and preventing them from relying directly on one another. This reduces dependencies within the system, making it easier to modify or extend individual components without causing issues elsewhere.

4.3 User Interface & Navigation

The user interface is designed to provide a simple and intuitive way for users to navigate the system and interact with the lesson content. Particular focus was placed on clarity and ease of navigation to support learning.

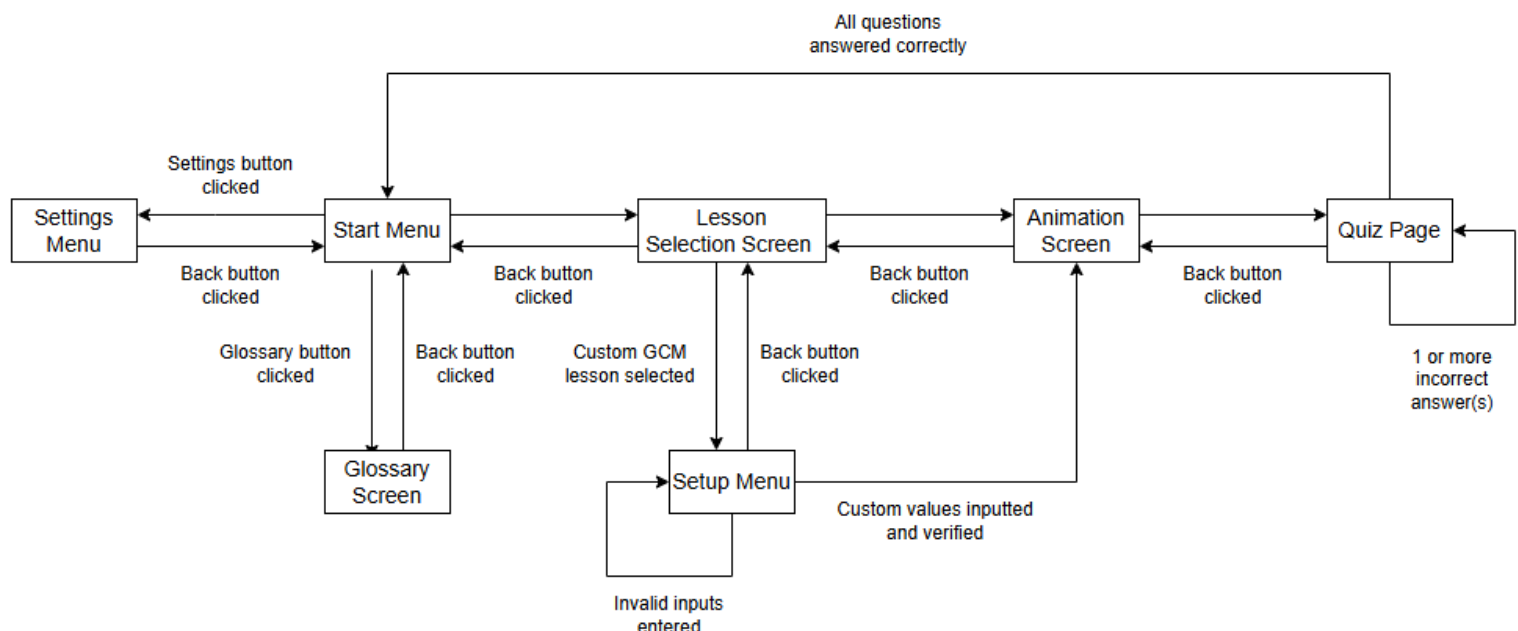


Figure 4.3.1 Navigation flow diagram

Figure 4.3.1 shows how users navigate between the main sections of the system. They start at the main menu and from that can select between the lesson page, settings and glossary. Once a lesson has been selected, the user is then taken

through the next stages of the user interface. The diagram also displays how all navigation actions are always reversible using a back button, this avoids 'locking in' of users to potentially long processes that they might have mistakenly clicked.

The following mock-ups show what these screens look like and how the interface is laid out.

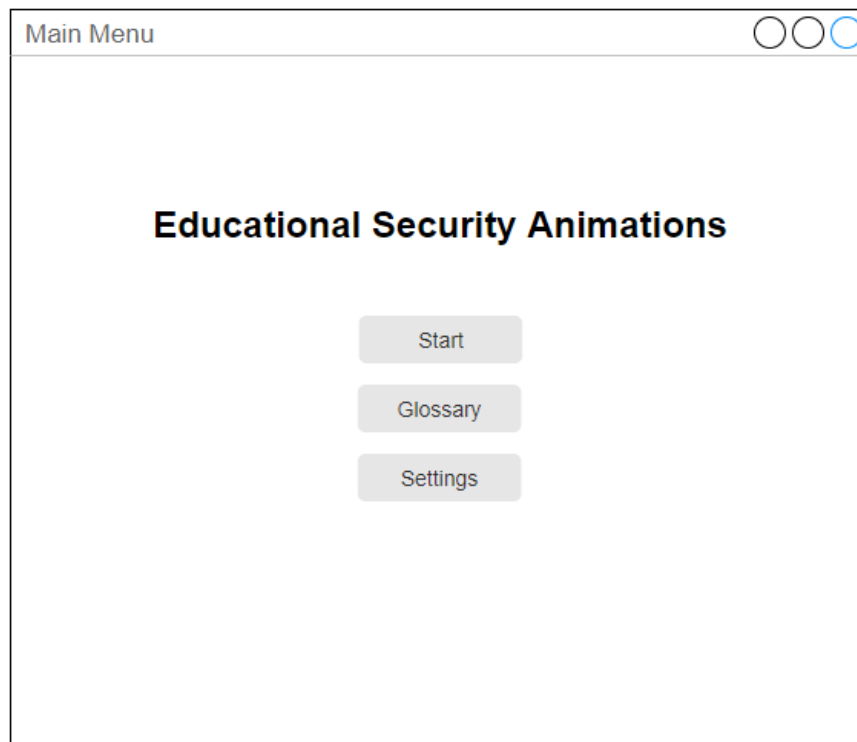


Figure 4.3.2 Main menu design

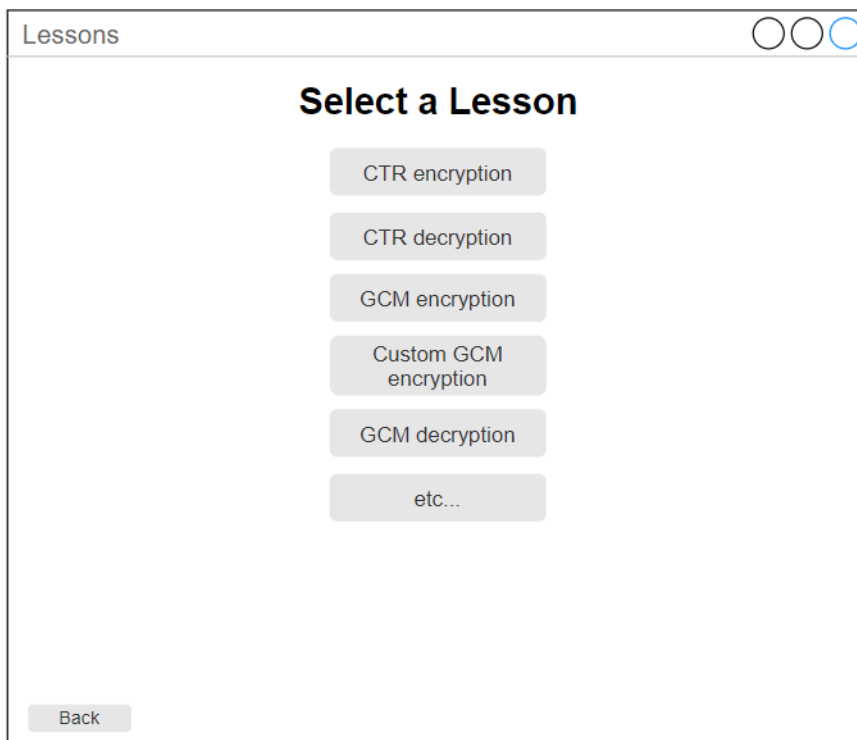


Figure 4.3.3 Lesson selection screen design

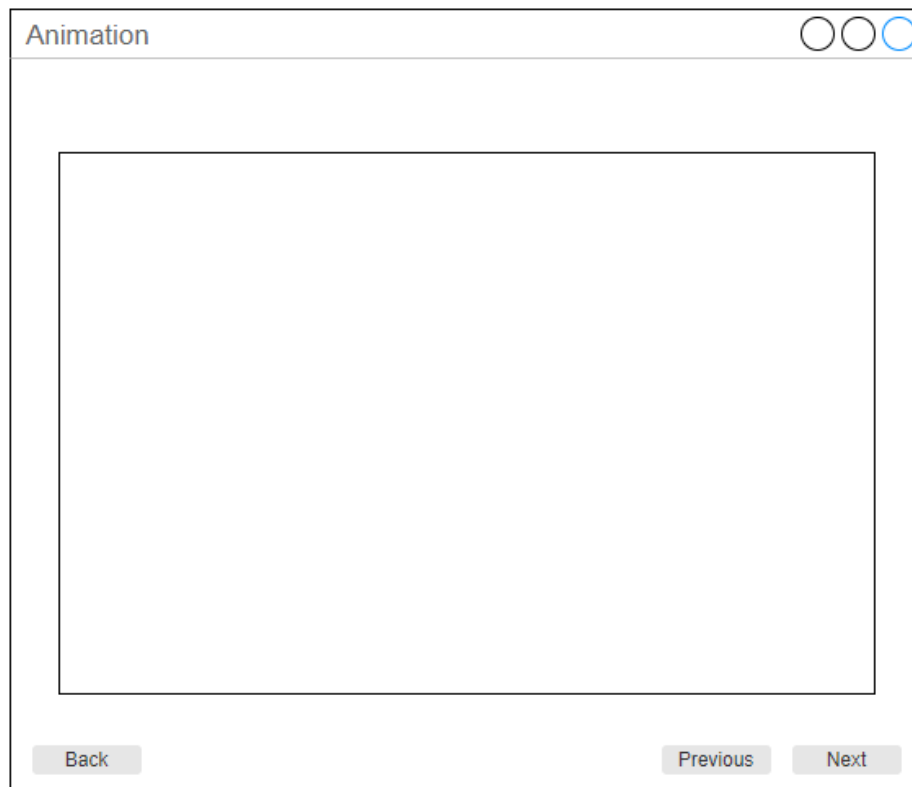


Figure 4.3.4 Animation screen design

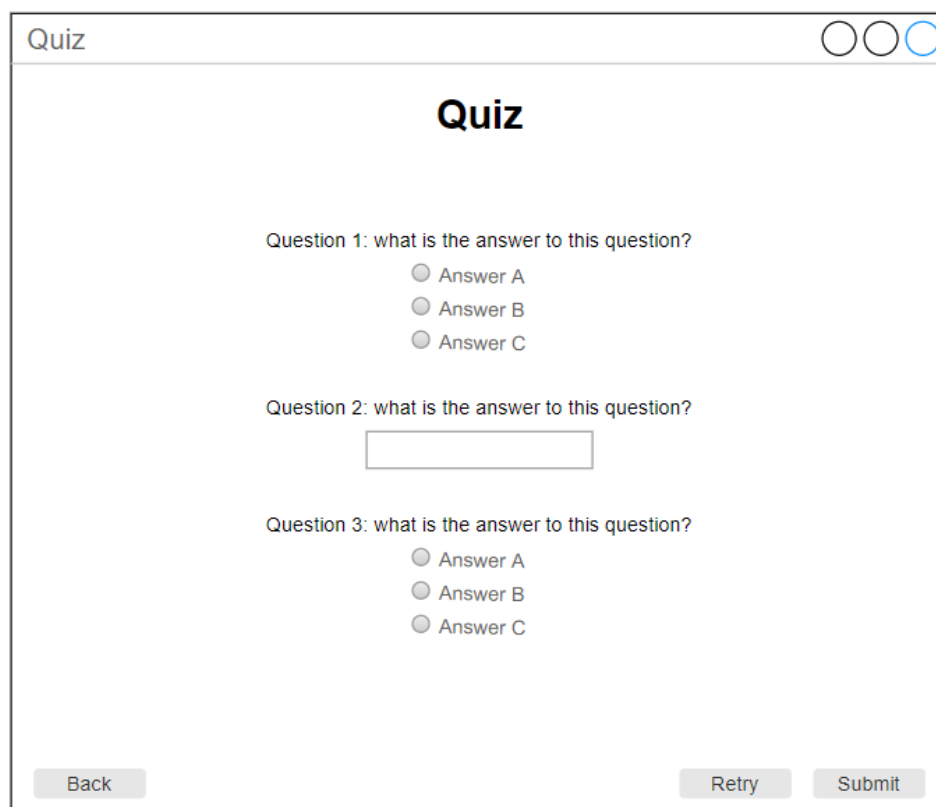


Figure 4.3.5 Quiz screen design

Figures 4.3.2 – 4.3.5 display the interfaces that the user will interact with in order to start the program and complete a lesson. Each interface except the main menu has a back button in the bottom left, allowing any navigation to be undone. The design

also centres content whenever possible, in figure 4.3.5 the quiz questions are centred on the page, and in figures 4.3.2 and 4.3.3 all buttons are stacked uniformly on the middle of the page. This improves clarity and ease of use, as centrally located elements naturally draw the user's attention, reducing the chance of distraction from less relevant interface elements. This layout also provides consistent visual structure across different screens, making the system more predictable and easier to use. Users do not need to search for key interactive elements.

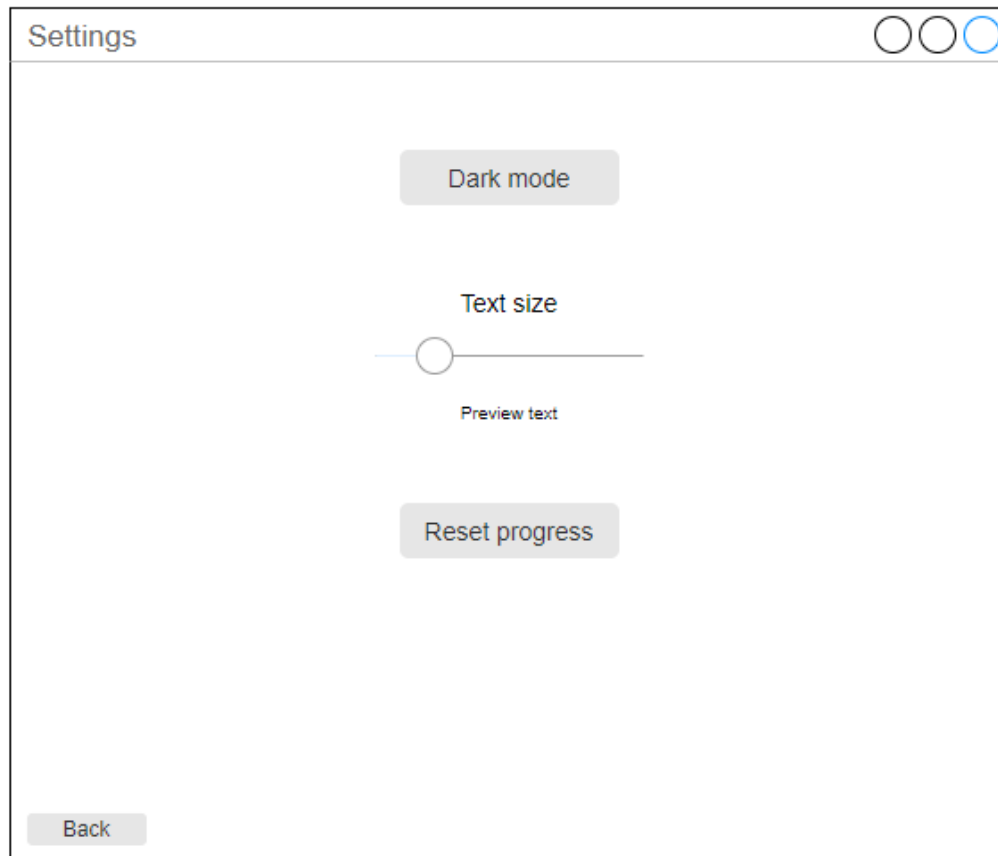


Figure 4.3.6 Settings screen design



Figure 4.3.7 Settings screen design

Whilst the settings and glossary screens are not part of the main program flow, both designs keep the consistency of back buttons, and the central design is applied to the settings screen.

The glossary page is designed with a two-panel, master-detail layout, where a list of terms is displayed alongside a panel showing the selected definition. This approach separates navigation from the content, allowing users to quickly locate terms. Keeping available options visible supports the principle of recognition over recall, defined in Nielsen's heuristic #6 (Nielsen, 1994), reducing the need for users to remember terms while navigating the system.

This design also reduces cognitive load by allowing users to switch between terms without changing pages improving usability (Sweller, 1988). The consistent layout and persistent navigation panel improve usability further by ensuring predictable interaction.

4.4 User Interaction

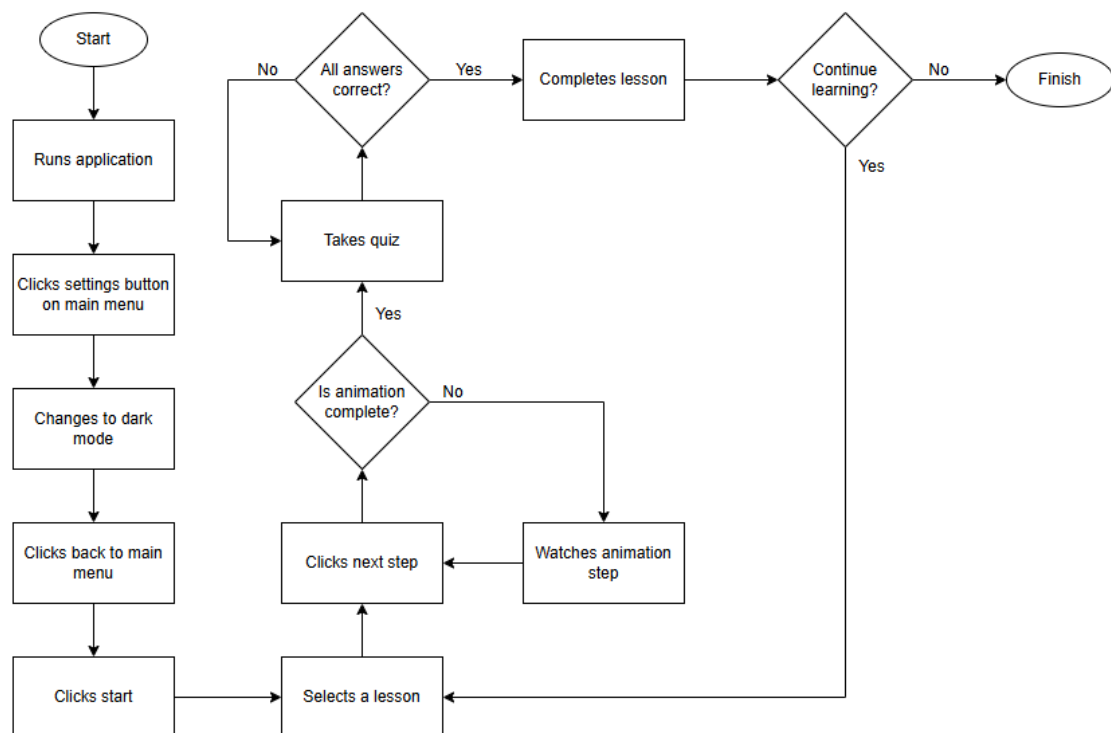


Figure 4.4.1

Figure 4.4.1 illustrates a typical use case for the system. The user begins at the main menu which acts as a central hub from which all major features of the system can be accessed. The user then navigates to the settings menu to switch to dark mode, then navigates back to the main menu.

From the main menu, the user clicks start and selects a lesson, which transitions the user to the animation interface. The user clicks the next step button on the animation page, triggering a decision, if the animation is not completed, the next step is displayed, and the system waits for further input. If it is completed then the user takes the associated quiz, which assesses their understanding of the material.

After submitting their responses, they are marked by the system, and if all questions are answered correctly, the lesson is complete, if they are not, the quiz is retaken. Finally the user has the choice to continue learning by selecting another lesson to complete.

This user journey through the system was designed to be intuitive by creating simple and logical steps between features. The system is also designed to uphold consistency, for example, every lesson ending with a quiz that must be completely correct before proceeding, which helps the user to understand the system and know what behaviour to expect from it.

4.5 Lesson Design

The design of the lessons focuses on ensuring that content remains understandable. Cryptographic algorithms are naturally complex, which is what makes them secure, but this also makes them challenging to learn. To address this, each algorithm is broken down into small, more manageable steps so that users can follow the process more easily. The GCM encryption lesson is used here as an example to demonstrate this approach.

Figure 4.5.1 shows the algorithm of GCM encryption, displaying how ciphertext and a corresponding authentication tag is created from plaintext, an IV and the key. This was then used as the basis from which the decomposition was created. Figure 4.5.2 demonstrates how this algorithm is taught in the system. It is decomposed into its core sections, and then each of these key sections are explained through both animations and text. This allows the complex mode of operation to be presented in a structured and accessible way.

This approach reflects the objective set out in Section 3.2 of simplifying cryptographic concepts. Enabling users to learn about a single part of the algorithm without being overwhelmed by the algorithm as a whole.

This design methodology was repeated for every other lesson added to the system. Each algorithm was first analysed and decomposed in order to translate them into lesson steps. Whilst the content of lessons varies, the step-based structure and quiz assessment remains consistent throughout. Due to this similarity of structure, only GCM has been presented, additional examples are provided in Appendix D.

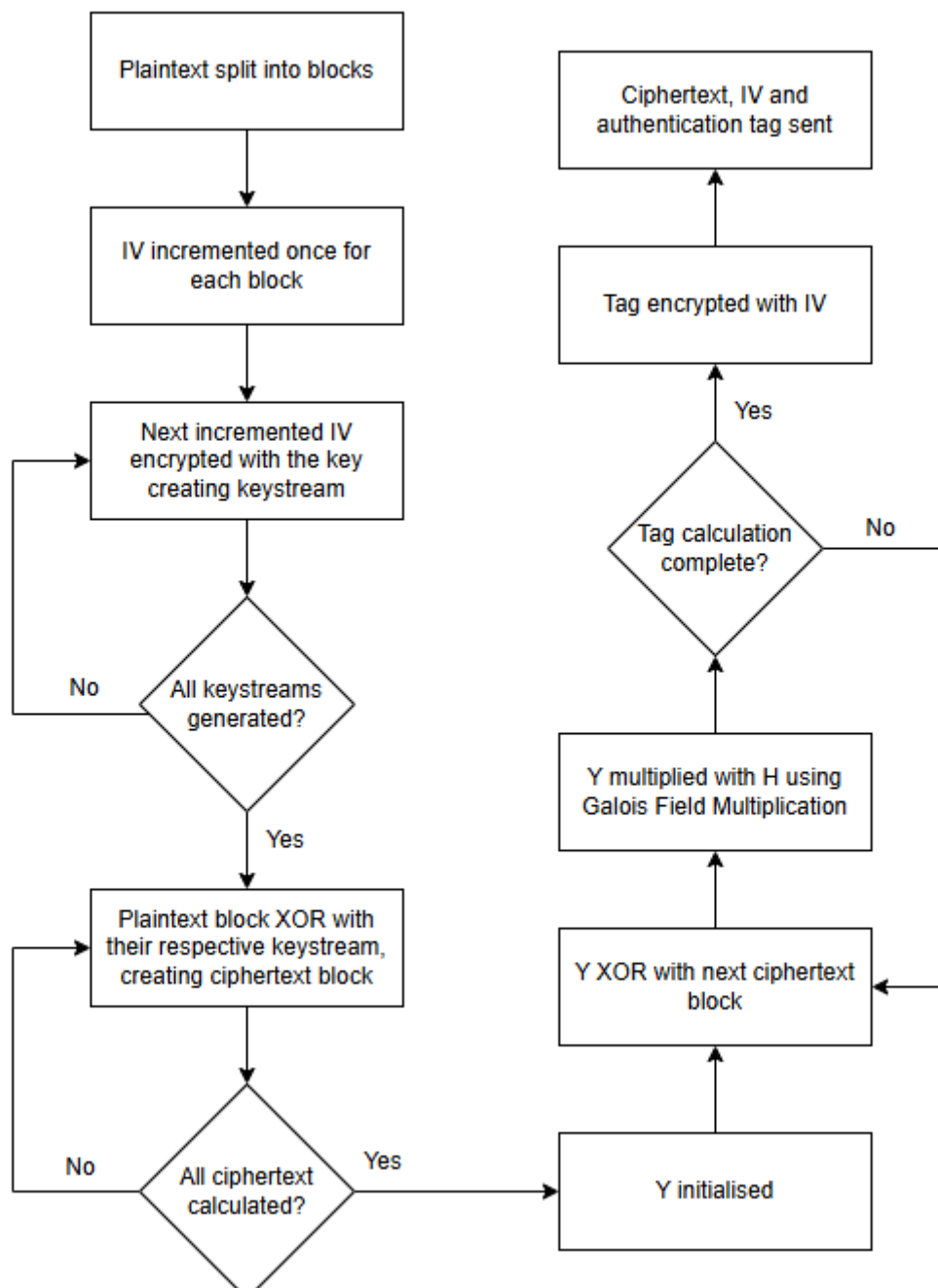


Figure 4.5.1 GCM encryption algorithm

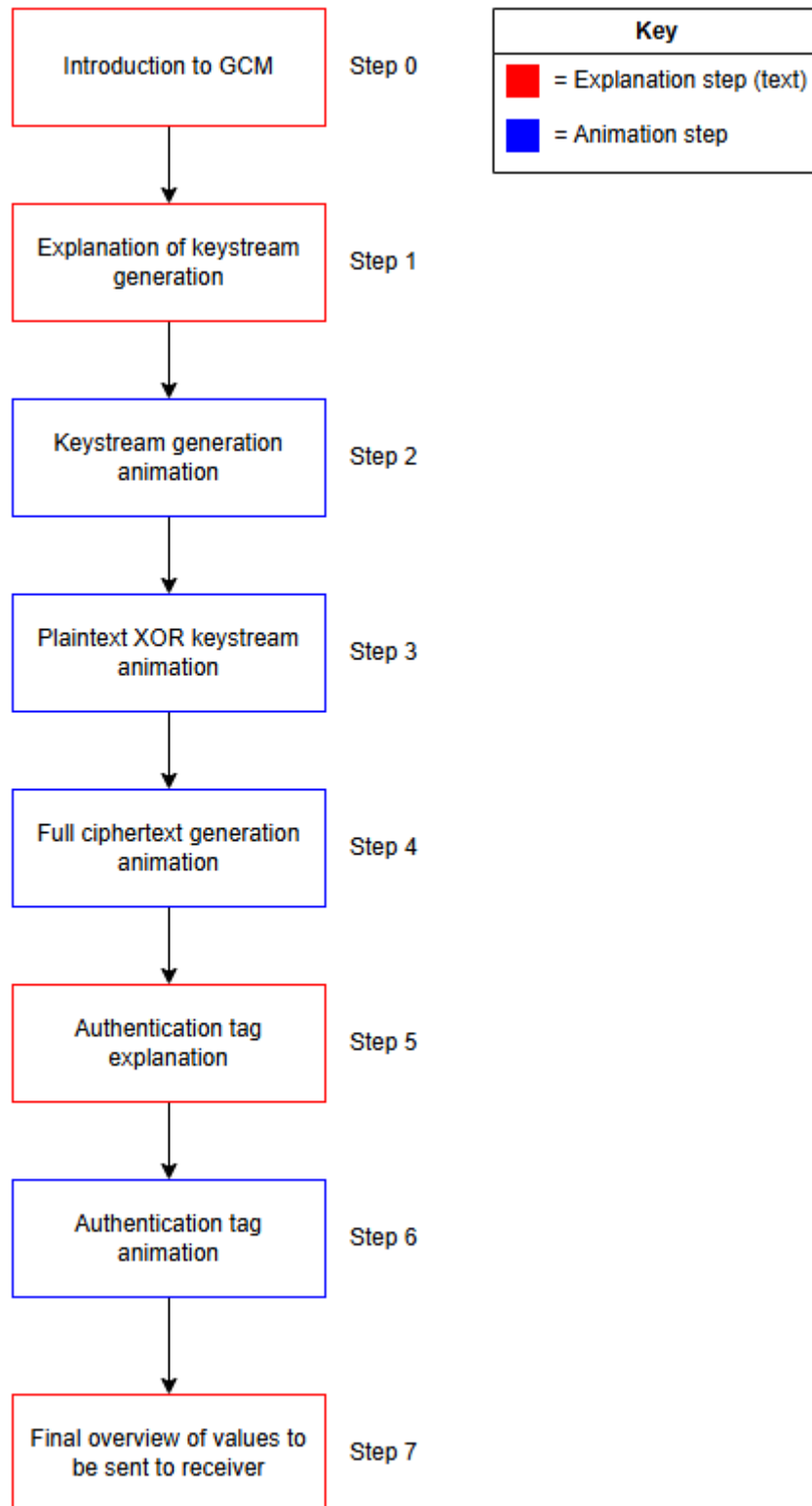


Figure 4.5.2 Steps for GCM encryption animation

The mixture of text-based and animation-based steps allows context to be established at the start of the lesson, as well as allowing for complicated steps to first be explained in text, as a primer, before being taught through an animation. This increases the likelihood that the user will understand what is being shown in the animation and its significance to the algorithm as a whole.

4.6 Data Flow & Storage

There are various flows of data throughout the program, these flows, as well as storage of data is shown in Figure 4.5.6. This diagram shows how data moves between the user, various core processes, and persistent data stores. The user acts as the primary external entity which provides input to the system through actions such as selecting a lesson, submitting quiz answers, accessing the glossary and modifying settings.

The animation system receives lesson related input and creates a visual output based on the cryptographic algorithm that the user selected. The quiz system receives answers and processes them to mark them, producing progress data that is stored in progress.txt.

The settings process handles user preferences for display settings. Settings data is written to and read from settings.txt, allowing user selections to persist across sessions. Similarly, the glossary process retrieves data from glossary.txt and presents it to the user when requested.

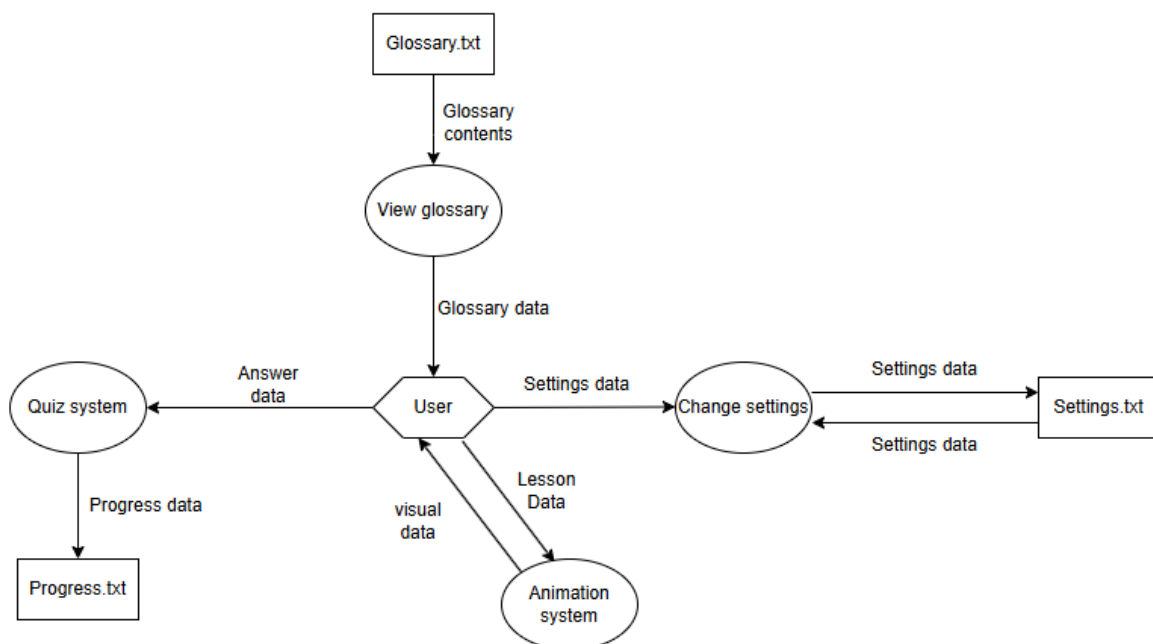


Figure 4.6.1 Data flow diagram

4.7 Design Constraints

The design of the system was shaped by a number of constraints, such as scope, time, and technical limitations. These constraints changed aspects of the system, these impacts are discussed in this section.

Time was one of the biggest constraints on the system. The process of creating lessons and building a user interface was likely to be highly time-consuming, and so the design had to take this into account through carefully considering what lessons to include in the final system. Time also limited the number of extra/accessibility features that could be considered.

Availability of libraries and other examples of similar systems was another constraint affecting design decisions. For example, to implement a lesson involving custom values inputted by the user, a version of the algorithm being taught needed to be created in order to calculate the correct values. Already existing python libraries, such as PyCryptodome, were insufficient for this use case, as they are focused on real world applications of these cryptographic algorithms. In order to keep lessons clear and readable, the algorithm must have a level of abstraction to ensure that the user can understand the basic concepts, as the goal of the system is to teach the theory, not practical implementation of the algorithm. These libraries required realistic values, such as a block size of 128 bits. This many bits would be unreadable on an animation and so the decision was made to have the lessons show the algorithms with a block size of 4 for the sake of visual clarity. The result of this constraint is that a custom 'toy' GCM mode of operation implementation had to be developed. These libraries also returned only the result of the algorithm, whereas animations need all intermediate values that lead up to the final result so that the process can be visualised.

Availability of acceptable animation libraries was another constraint that influenced design decisions and project development planning. Whilst there are some libraries that extend Tkinter with animation capabilities, such as tk-animations and PyTkAnim, these are primarily designed for simple visual effects rather than longer animation sequences. Additionally, there is also a lack of documentation and tutorials for these libraries, resulting in them being likely to increase complexity rather than reducing it. As a result, a custom animation framework had to be designed.

The target audience of students beginning to learn these concepts also acted as a constraint on the design. Lessons about these cryptographic algorithms had to be designed in such a way that they were simplistic enough to be understandable by an average computer science student, but also in depth enough to teach the algorithm accurately, without a large amount of abstraction obscuring the actual process. This also influenced decisions about what features to add, as any extra feature must serve a purpose of supporting a student's learning.

The chosen user interface library Tkinter created a constraint on the complexity of the user interface. Whilst Tkinter is highly popular, it is widely used for more simplistic user interfaces, this is due to the fact that other user interface frameworks, such as the originally chosen Qt, are available that are more suitable for more complicated user interface designs and interactions. Therefore, user interfaces had to be designed in a way that reduced the number of widgets that are on screen and kept layouts simple to program.

Implementation

5.1 Implementation Overview:

The system was implemented using Python and Tkinter as the graphical user interface framework following the design outlined in Section 4. The implementation makes use of a modular structure, with functionality divided across multiple files, including a central control file, individual lesson files, and shared utility files that are imported where they are required.

The animation system was implemented using a step-based approach, allowing users to progress through small stages of the algorithm and build an understanding of it incrementally. Persistent data, such as user progress and settings, is managed using simple text files.

During development, several challenges were encountered and overcome. These are discussed further in Section 5.5.

5.2 User Interface:

The user interface was implemented using Tkinter according to the design described in Section 4. This section explains how this interface was realised and how key user interface logic such as navigation is handled.

The design of the user interface makes use of many different 'screens' such as the main menu, lesson selection screen, glossary screen, etc. In order to implement this, these screens were created as separate pages, which are then displayed over the top of a plain static background. This approach ensures that all user interface pages are encapsulated from one another and has the added benefit of ensuring that the sizing of the program window stays consistent throughout, as that is defined higher up the hierarchy. Figure 5.2.1 gives an example of how these pages were implemented and stored to enable navigation later on.

```
243 # Creating start page
244 start_page = tk.Frame(container, bg=cf.Colours["bg"])
245 start_page.place(relx=0, rely=0, relwidth=1, relheight=1)
246 pages["start"] = start_page
247
248 center_frame = tk.Frame(start_page, bg=cf.Colours["bg"])
249 center_frame.place(relx=0.5, rely=0.5, anchor="center")
250 title_label = tk.Label(center_frame,
251                        text="Educational Security Animations",
252                        bg=cf.Colours["label_bg"],
253                        fg=cf.Colours["text"],
254                        font=cf.Fonts["intro_screen_text"])
255 title_label.pack(pady=(0, 48))
256 btn_stack = tk.Frame(center_frame, bg=cf.Colours["bg"])
257 btn_stack.pack()
```

Figure 5.2.1 Start page code section

Navigation was handled by storing each of the pages in a list every time one was created. At run time, this list is populated with all of the pages in the system. A small function called `show_page` (Figure 5.2.2) is then called whenever a page switch is needed, for example, when the user presses the start button. This lifts the requested page to the top, making it visible and interactable. This function was then referenced as the command for certain button presses, allowing button clicks to cause the window to change appearance.

```

25     # Page switching logic
26     pages = {}
27     def show_page(name): 11 usages
28         pages[name].lift()

```

Figure 5.2.2 Page switching logic

Each of the pages, except the main menu, were designed with back buttons, which allow the user to navigate to the previous page, for example, the back button on the lessons page navigates to the main menu. This ensures responsive and intuitive navigation across all pages of the user interface.

Although all pages of the user interface were designed as shown in section 4, some alterations had to be made due to issues relating to Tkinter as well as ones that the designs did not foresee. The glossary page (Figure 5.2.3) is one example of this.

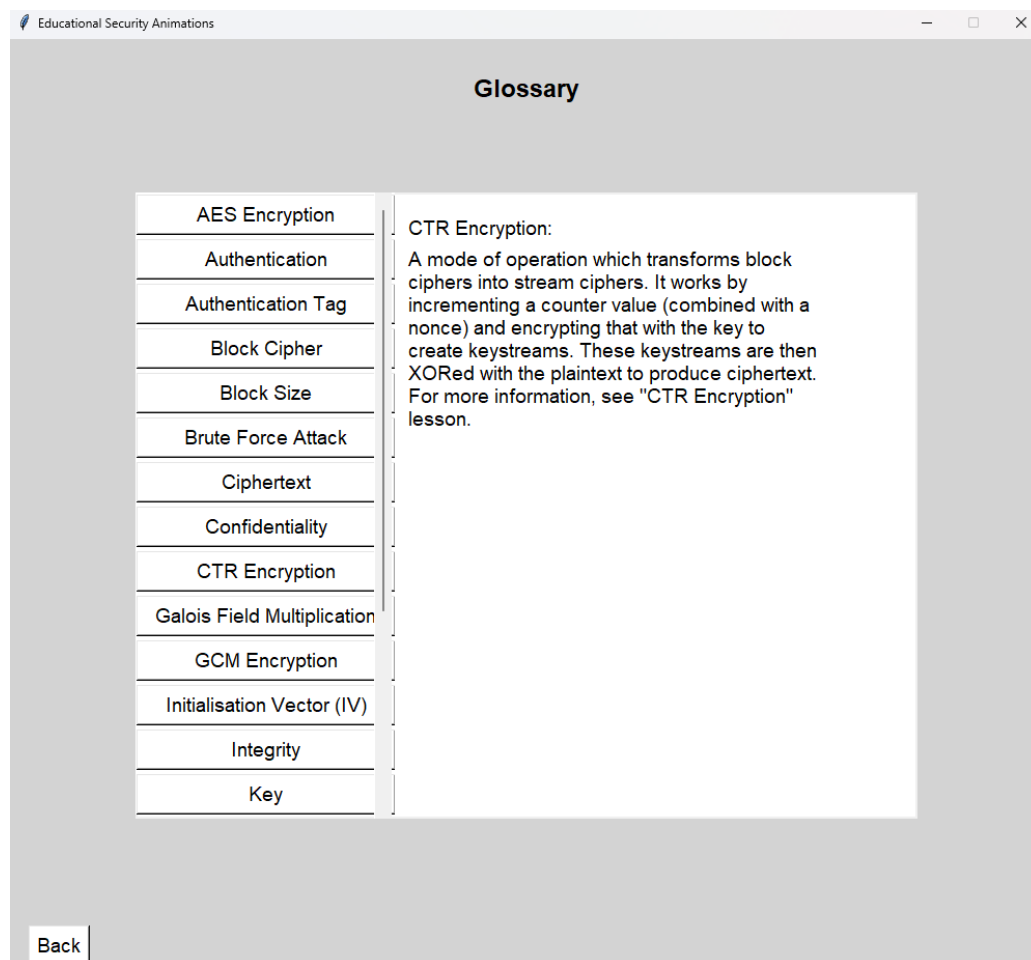


Figure 5.2.3 Glossary page

The original design contained a list of buttons for each glossary item, however when implementing the glossary, too many terms were added, which resulted in them overflowing the container. To fix this, a scroll bar was added to the right of the buttons allowing the user to access items at the bottom of the glossary.

Another change that had to be made was the addition of an apply button to the text size change slider on the settings page (Figure 5.2.4). The original design intended for the slider to only affect the preview text at first and change text sizing globally if the user left the slider in position for a set amount of time. This design choice was to reduce the number of widgets on screen for simplicity. However, when implementing this, the slider caused issues by updating all text sizes at once, which was disorienting, especially if the slider was moved quickly. Therefore, the button had to be added, so that only the preview text changed as the slider was moved, and changes did not apply globally until the user clicked it. An added benefit of this is that it also reduces the likelihood of accidental changes, as it is now a two-step process.

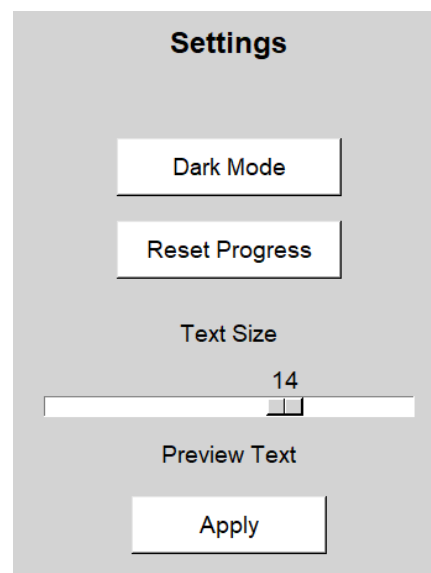


Figure 5.2.4 Section of settings page

Other minor changes were made such as adding a step counter to the animation page, as during debugging, it was often difficult to know which step was causing a bug. This also functions to give an indication of progress throughout a lesson for the user. Background colours were also tweaked to guide the user's attention to specific UI elements, e.g. the background of the animation page is light grey, whereas the animation canvas is white which draws attention to it.

5.3 System Structure:

The system is a modular python application, with code split across multiple files, with each file having a clear role in the wider function of the system.

Figure 5.3.1 shows the structure of the system's codebase. Main.py is the central control module handling starting the program, creating the user interface, controlling navigation between screens, and handling user interactions with widgets.

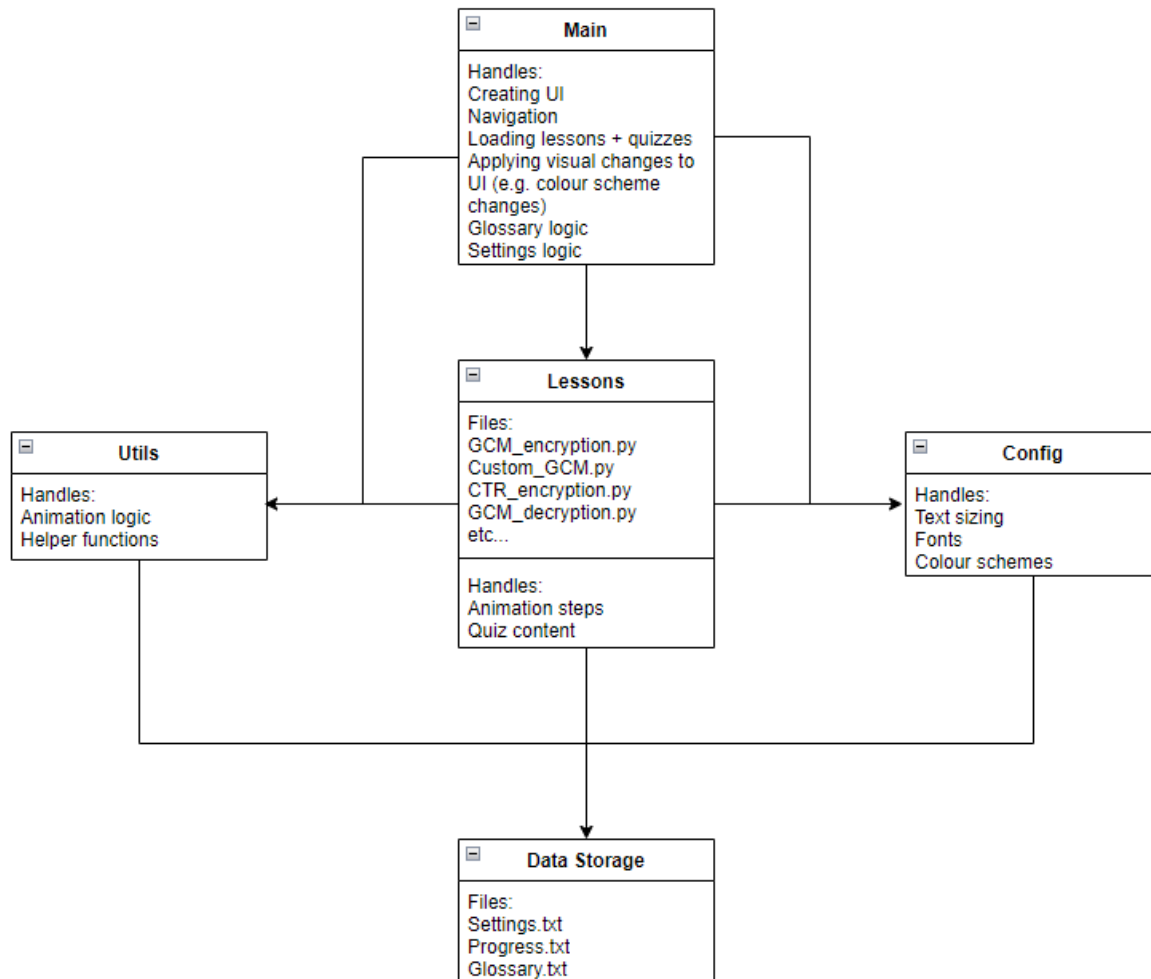


Figure 5.3.1 Diagram of codebase structure

Each lesson implemented in the program is its own separate file, which contains the animation steps, quiz questions and their respective answers, as well as a function to return the list of steps to main.py. This results in each lesson being independent of each other, meaning that new lessons can be added without disrupting already existing ones, and debugging a specific lesson is much simpler.

Utils.py contains reusable helper functions that are used across multiple other files, its main role is storing the animation functions, but other helper functions are also contained within it. This file is then imported into every other file, avoiding high volumes of repeated code and assisting with debugging, as each function only has to be tested and implemented once.

Config.py stores information required to create the user interface. It stores the colour schemes for light and dark mode, fonts, and text sizes. This file is referenced whenever a widget is created or reconfigured which allows the styling to stay consistent and support features such as theme switching.

Three data files are used to act as persistent storage, all of which are read from at runtime. Glossary.txt is used to populate the glossary page with relevant terms and definitions. Settings.txt and Progress.txt are both written to during runtime, which

allows for saving of settings preferences and user completion of lessons. Examples of these text files can be seen in Appendix C.

When a user selects a lesson, the main file loads that lesson module. The lesson provides the animation steps and the quiz content, using the utility functions for animation behaviour and rendering of objects. Once the quiz is complete, the progress file is accessed and appended.

This modular system structure supports the extensibility of the program, as new lessons and features can be developed using the already existing structure and functionality. Separating lessons and features allows for easier maintenance, and reusable functions cut down on duplication, reducing file sizes and increasing readability of code.

5.4 Key Components:

The animation system is a central component of the system; it is responsible for visualising the cryptographic algorithms in a structured and understandable way. Due to Tkinter's lack of native animation support, a custom animation framework had to be created. This was achieved by creating a function to draw blocks on screen (Figure 5.4.1), and assign them tags, so that they could be moved, recoloured, and have text adjusted later.

```
def draw_block(canvas, root, x, y, text, outline, block_size, tag):
    # Draws rectangle that is centred around x,y
    c = get_dark_mode()

    if block_size == 1:
        x1 = x - 60
        y1 = y - 25
        x2 = x + 60
        y2 = y + 25
    else:
        x1 = x - 15
        y1 = y - 12
        x2 = x + 15
        y2 = y + 12

    canvas.create_rectangle(x1, y1, x2, y2, outline=outline, tags=(tag, tag + "_rect"), width=2)
    canvas.create_text(x, y, text=text, font=cf.Fonts["block_text"], fill=c["text"], anchor="center", tags=(tag, tag + "_text"))
```

Figure 5.4.1 Block drawing logic

Small movement functions (Figure 5.4.2), that can be imported and called by the lesson files, were then created in order to facilitate the movement of the blocks once they had been created. Diagonal movement was also facilitated through calling vertical and horizontal movement functions simultaneously.

```

def move_block_up(canvas, root, tag, target_y):
    coords = canvas.bbox(tag)
    current_y = (coords[1] + coords[3]) / 2
    if current_y > target_y:
        canvas.move(tag, 0, -2)
        root.after(10, lambda: move_block_up(canvas, root, tag, target_y))
    else:
        return

def move_block_right(canvas, root, tag, target_x):
    coords = canvas.bbox(tag)
    current_x = (coords[0] + coords[2]) / 2
    if current_x < target_x:
        canvas.move(tag, 2, 0)
        root.after(10, lambda: move_block_right(canvas, root, tag, target_x))
    else:
        return

```

Figure 5.4.2 Section of animation logic

Although these movement functions could have been generalised into a single one, by adding a direction vector as a parameter and then adjusting the variables accordingly, they were intentionally implemented as separate functions to prioritise readability of animation code. Each function clearly labels its purpose, reducing the cognitive load when developing and debugging the animation steps. Given the small size of the functions, the code duplication is minimal therefore there is no significant impact on the maintainability of the system.

Quizzes were another core component to the educational design of the system. They appear after the animation steps have all been completed and ensure the user can answer questions about lesson content correctly. Questions and their respective answers, as well as the correct answer, were stored in a dictionary in lesson files (for examples see Appendix B). This dictionary was accessed as the lesson was loaded and then used to populate the quiz interface. When the function to change step detected that the user had finished the lesson, it called the `load_quiz` function (Figure 5.4.3). This function takes the dictionary as an input and then dynamically builds the user interface by creating a question label and linked radio buttons for each of the questions. Once this has been built, a submit button is placed on the interface.

```

def load_quiz(questions):
    global current_quiz, answer_vals, question_frames
    current_quiz = questions
    answer_vals = []
    question_frames = []
    c = get_dark_mode()

    for widget in quiz_canvas.winfo_children():
        widget.destroy()

    # Building container for each question and its answers
    for i, item in enumerate(questions):
        question_frame = tk.Frame(quiz_canvas, bg= c["bg"])
        question_frame.pack(pady=15, anchor="w", padx=40)
        question_frames.append(question_frame)
        question_lbl = tk.Label(question_frame, text=f"Q{i+1}- {item['question']}", bg= c["label_bg"],
                                fg=c["widget_fg"], font=cf.Fonts["label"], wraplength=650, justify="left")
        question_lbl.pack(anchor="w")
        value = tk.IntVar(value=-1)
        answer_vals.append(value)

        for j, answer in enumerate(item["answers"]):
            radio_button = tk.Radiobutton(question_frame, text=answer,
                                          variable = value, value = j,
                                          bg= c["label_bg"], fg=c["widget_fg"],
                                          font=cf.Fonts["body"], anchor="w",
                                          selectcolor= c["widget_bg"], activebackground= c["label_bg"],
                                          activeforeground= c["widget_fg"])
            radio_button.pack(anchor="w", padx=20)

    submit_btn = tk.Button(quiz_canvas, text="Submit", command=submit_quiz,
                           bg= c["widget_bg"], fg=c["widget_fg"],
                           padx=20, pady=5, cursor="hand2")
    submit_btn.pack(pady=20)
    show_page("quiz")

```

Figure 5.4.3 Quiz building logic

The interface then waits for the user to click submit, which triggers the code shown in Figure 5.4.4. This function calculates the score by comparing the ID of the selected radio button for every question to the ID of the correct answer. If the user did not get all of them right, or did not answer a question, a try again button will be displayed, allowing them to reset the quiz, which is achieved by again calling the code in Figure 5.4.3. If all of them are correct, the quiz ends and the lesson is marked as completed in a progress text file. An example of a complete lesson flow can be found in Appendix E.

```

def submit_quiz():
    score = 0
    c = get_dark_mode()
    for i, item in enumerate(current_quiz):
        if answer_vals[i].get() == item["correct*]:
            score += 1

    tk.messagebox.showinfo(title="Result", message=f"You scored {score}/3")
    for widget in quiz_canvas.winfo_children():
        if isinstance(widget, tk.Button) and widget.cget("text") in ["Finish", "Try Again*]:
            widget.destroy()

    if score == 3:
        finish_btn = tk.Button(quiz_canvas, text="Finish",
                                command= lambda: (mark_as_complete(current_lesson_name), update_lesson_buttons(), show_page("lessons")),
                                bg= c["widget_bg*], fg=c["widget_fg*], padx=20, pady=5, cursor="hand2")
        finish_btn.pack(pady=5)
    else:
        Try_Again_btn = tk.Button(quiz_canvas, text="Try Again*",
                                    command = lambda: load_quiz(current_quiz_questions),
                                    bg=c["widget_bg*], fg=c["widget_fg*], padx=20, pady=5, cursor="hand2")
        Try_Again_btn.pack(pady=5)

```

Figure 5.4.4 Quiz marking algorithm

Loading the lessons was simple to implement due to the design of the lesson files. Each lesson file was had a `get_steps` function which had all animation steps enclosed and returns a list of the steps. The loading function then took these steps and displayed the first one, as well as initialising a counter that tracks how far through the animation a user is. This component, as well as the animation framework and quiz logic, created the structure which allowed lessons to be implemented.

One of these lessons that were implemented in the system was GCM encryption (algorithm explained in Section 4.5). Figure 5.4.5 shows an example of the animation steps, that makes use of the movement functions discussed previously.

```

def step_4():
    col_width = W / 3
    delay = 0
    canvas.delete("all")
    info_lbl.config(text="")
    lock_buttons(next_btn, prev_btn)
    canvas.create_text(W / 2, 30, text="Encryption", font=cf.Fonts["title"], fill=c["text"])
    info_lbl.config(text="The other plaintext blocks are XORed with their respective keystream\n"
                        "giving us the following full ciphertext")

    for i in range(3):
        x_center = col_width * i + col_width / 2
        text = "Ciphertext Block " + str(i+1)
        canvas.create_text(x_center, 250, text=text, font=cf.Fonts["label"], fill=c["text"])

    for i in range(3):
        x_center = col_width * i + col_width / 2
        root.after(delay, lambda x = x_center, txt = Full_CT[i]: draw_block(canvas, root, x,
                                                                              y: 300, txt, cf.Block_colours["ciphertext*],
                                                                              block_size: 1, tag: "CipherText*))
        delay = delay + 1000
    root.after(3000, lambda: unlock_buttons(next_btn, prev_btn))

```

Figure 5.4.5 Example animation step code

Animation steps were created using the `root.after()` command, which delays the execution of code by a specified time, allowing drawing and movement of objects on screen to be timed precisely.

Several lessons were then created in order to teach the user about a variety of cryptographic functions, these were:

1. GCM encryption
2. GCM decryption
3. GCM encryption with custom input

4. CTR encryption
5. CTR decryption
6. The security problem
7. XOR

Lessons 6 and 7 were added to act as fundamentals lessons. All other lessons rely heavily on concepts taught in these lessons, and so they were implemented in order to ensure that the user had a strong understanding of this before proceeding on to the later, more complex lessons. They were also placed at the top of the lessons page (Figure 5.4.6), to ensure that users complete them first.

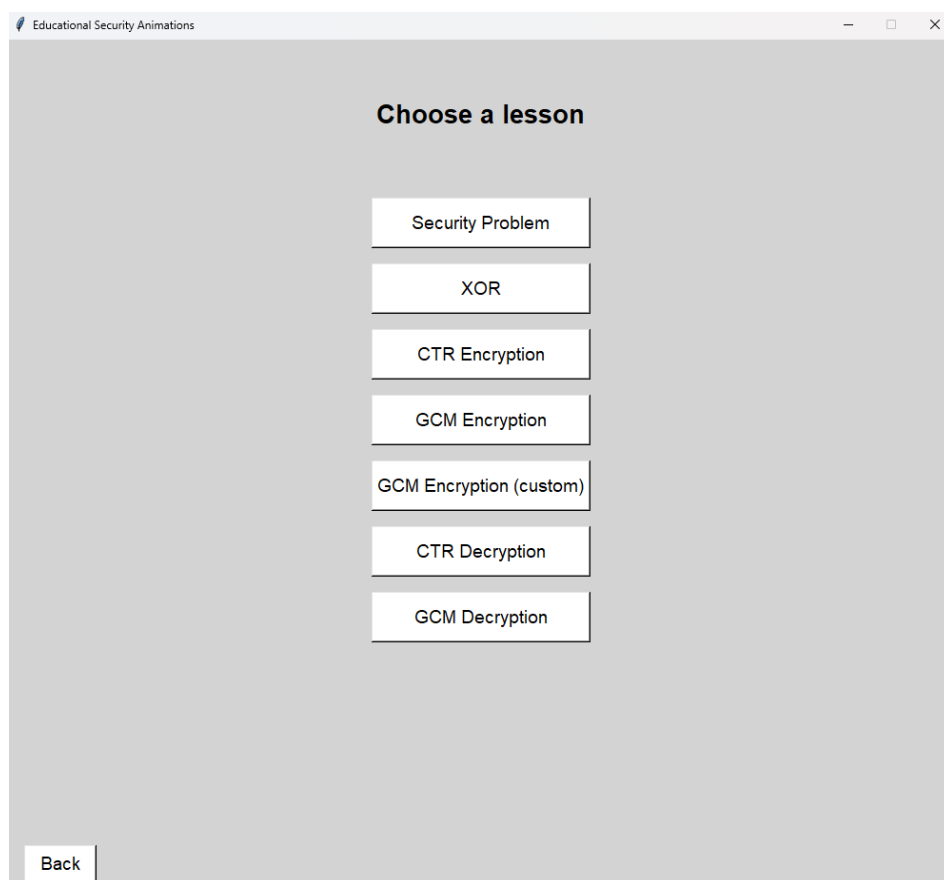


Figure 5.4.6 Select lesson interface

Lesson 3 was implemented to add an element of user interaction with the lesson. The user is prompted to enter custom values for the system to use in a GCM encryption lesson, allowing the user to experiment and gain deeper understanding. However, creating this lesson required a custom GCM implementation to be built (see Section 4.5). This 'toy' GCM returned lists of all intermediate and final values of the IV, keystreams, ciphertext and authentication tag. The custom GCM lesson then used these values when teaching the GCM encryption algorithm.

Tracking progress through these lessons was handled using a text file, which, upon completion of the quiz, was written to, indicating that the lesson was complete. Progression through the lessons is then indicated to the user through recolouring of

lesson buttons, which is achieved by reading this text file and parsing through the contents to see which lessons have been completed.

External data storage was also utilised to develop the glossary, which is parsed to extract terms and definitions, and then the list of terms is dynamically created based on the contents of the text file (Figure 5.4.7).

```
# Fetching glossary entries
entries = glossary_content.split("***")
glossary_list = []
for entry in entries:
    if "/" in entry:
        items = entry.split("/")
        term = items[0].strip()
        definition = items[1].strip()
        glossary_list.append([term, definition])
glossary_list.sort(key=lambda x: x[0].lower())

def show_definition(term, definition):
    definition_label.config(text=definition)
    term_label.config(text=term + ":")

# Create buttons for each term
glossary_btns = []
for item in glossary_list:
    c = get_dark_mode()
    term = item[0]
    definition = item[1]

    btn = tk.Button(term_frame, text=term, anchor="center", bg=c["widget_bg"], fg=c["widget_fg"],
                    command=lambda t=term, d=definition: show_definition(t, d), cursor="hand2")
    btn.pack(pady=2, padx=1, fill="x")
    glossary_btns.append(btn)

term_frame.update_idletasks()
term_canvas.configure(scrollregion=term_canvas.bbox("all"))
```

Figure 5.4.7 Populating glossary code

Text files were also key in the development of accessibility features, allowing settings to be ‘remembered’ by the system. Adjustable text size was implemented by modifying a text size variable in the configuration file, which then triggers an update to all widgets globally.

Dark mode functions through a variable which acts as a toggle between light and dark modes. Before any widgets are drawn, a function is called to fetch the status of this variable, and then based on this status, change which of the two themes are used to draw these variables.

```

Colours = {
    "bg": "light grey",
    "widget_bg": "white",
    "widget_fg": "black",
    "text": "black",
    "label_bg": "light grey",
    "complete": "light green"
}

Dark_colours = {
    "bg": "gray12",
    "widget_bg": "gray20",
    "widget_fg": "white",
    "text": "white",
    "label_bg": "gray12",
    "complete": "forest green"
}

```

Figure 5.4.8 Light/dark colour schemes

5.5 Challenges:

During implementation of the system, several challenges were faced that influenced both the development process and aspects of the final system. These challenges mainly stemmed from limitations of chosen technologies, and complexity involved with developing animations.

The original plan for the system involved using Qt designer for the user interface designs and animations. This technology was chosen due to the drag-and-drop style user interface design tool, as well as the level of control that it gives over the appearance and behaviour of the user interface. Whilst the user interface design tool was intuitive and allowed interfaces to be built quickly, when trying to implement animation logic, as well as when fixing bugs, the high level of control over the user interface became a drawback. The debugging process was made much longer due to this. Therefore, the decision was made to switch from Qt designer to Tkinter. This user interface framework was originally not chosen due to the lack of a simple user interface design tool, instead it has to be created in code. Tkinter also lacks built in animation functionality, so a custom animation engine had to be built in order to facilitate animations (Figure 5.4.2). Once this engine was built, the process of creating animations was much faster, and the debugging process more efficient. Creating animations was the most time-consuming element of the implementation process due to the high volume of challenges and bugs that it generated. One of the main issues was animations playing over each other if the next button was clicked

whilst an animation was still in progress. Clearing the user interface at the start of each step of the animation was one fix that was implemented, which worked for any text-based steps, however, animations would still be drawn over each other. This is the result of how the animations were created. Each movement of the animation had to be programmed with a delay, this meant that if the next or previous buttons were pressed during an animation, the interface will begin showing the next step, however the previous step's code was still running, meaning that animations executed over the top of the current step. Therefore, buttons were locked during animations and unlocked after (Figure 5.5.1).

```
def lock_buttons(next_btn, prev_btn):
    next_btn.config(state="disabled")
    prev_btn.config(state="disabled")

def unlock_buttons(next_btn, prev_btn):
    next_btn.config(state="normal")
    prev_btn.config(state="normal")
```

Figure 5.5.1 Button locking/unlocking utility functions

Whilst this solution takes away control from the user temporarily whilst animations are running, it also ensures that the system stays consistent and avoids visual errors which could confuse learning. This solution also ensures that users are not skipping through the lessons without letting them play.

Another challenge of implementing animations was the coordinate system that is used for canvas objects in Tkinter. Instead of acting like a graph, where the origin is in the bottom left corner, canvas objects have their origin in the top right corner. This caused objects to be drawn and moved to incorrect locations in space. This challenge was overcome by changing how animations were designed before implementation to consider the non-intuitive coordinate system.

Tkinter delays being inconsistent and not precise also caused issues for the animation system. If timings of animations were too close together, it often caused unexpected errors. This is due to the fact that the timings of the moving blocks were slightly inconsistent and could be influenced by things such as moving the window around or resizing it. The solution to this issue was to fix the window size and ensure that an animation had enough time to execute, so they would not overlap with the next animation, even if it was slightly delayed.

Implementing a dark mode toggle also presented more challenges than expected. The planned implementation required each widget to be stored in a list depending on its type so that it can be recoloured when dark mode is toggled on/off (see Section 5.4). However, many widgets were missed from this list causing inconsistent application of the dark mode. This was fixed through testing of every single widget to ensure it was behaving as expected. Dark mode toggling also caused issues with the progress tracking system. If lessons had been completed, their buttons should appear green on the lessons page to signify to the user that they are complete, however when switching to dark mode these buttons were being recoloured, making

the lessons appear incomplete. This was fixed by calling the function which recolours the buttons based on completion when dark mode was toggled on or off.

The original design for the quiz system included a variety of question types (e.g. multiple choice, true/false, typed answer, etc.) this raised issues with the user interface. This is due to the user interface being created dynamically when the quiz was loaded, which meant that varying question types added significant complexity to an already technical system, creating a large number of visual bugs. Therefore, the decision was made to simplify the design and limit quizzes to three questions, all of which are multiple choice questions with three possible answers.

5.6 Debugging methodology:

This section outlines the approach taken during implementation to identify and resolve errors. It focuses on debugging steps rather than formal intensive testing, which is discussed in Section 6.1.

The first important debugging strategy was incremental development. Each feature was developed one at a time and tested in isolation iteratively. This approach resulted in most issues being caught quickly. The next feature was only developed when the previous was tested and had no major bugs.

The next strategy was isolating issues once they had arisen. Quickly narrowing down on the troublesome code reduced diagnosis times. Isolating the issue also ensured that no working code was altered during the process of fixing a bug, only code that was not functioning as intended was changed.

These two debugging strategies were achieved through the use of temporary logging using print statements to identify unexpected variable values. This then allowed the problem code to be located. Step by step execution was also employed, which was particularly useful when debugging the animation stages.

AI tools, ChatGPT and Claude, were also used during development as debugging aids. These tools were used to identify issues and suggest methods for fixes. Any output from these tools was not assumed to be accurate and was always validated through testing the system after the fix had been deployed. Suggested solutions often had to be modified in order to work within the system, especially user interface adjustments. These tools were used solely as assistive devices during debugging, all design decisions, system architecture and final implementations were not the result of AI tools.

Results & Evaluation

6.1 Testing Approach

Testing was carried out during development using an iterative approach, with individual components tested as they were implemented. This approach allowed issues to be identified and fixed early, especially in more complex systems such as the animations.

Key components such as the animation system, quiz logic, and file handling, were tested in isolation. This ensured that each part functioned correctly before being integrated into the wider system. Following development, end-to-end testing was conducted to validate complete user flows, such as progressing through a lesson, completing a quiz and updating progress.

Edge case testing was also conducted, including repeated unpredictable navigation through animation steps and modifying stored data to simulate corrupted files. This ensures stable behaviour under normal use. These tests were chosen to verify the requirements set out for the system in Section 3 and prove that it behaves reliably. Detailed test cases can be found in Appendix A.

6.2 Functional Requirements Evaluation

ID	Requirement	Test performed	Expected result	Actual result	Pass/Fail
FR1	Animation of cryptographic algorithms	Navigate through full animation sequence	All steps display correctly	All animation steps displayed in correct order, no inconsistencies observed	Pass
FR2	Supported algorithms	Open each lesson required by FR2 and verify each loads the correct content	Correct animation and content loads for each	All lessons loaded and correct content was displayed	Pass
FR3	Quiz functionality	Complete quiz with correct answers and submit	System displays results immediately	Feedback was displayed immediately after submission, accurately identifying correctness of answers	Pass

FR4	Animation control	Click 'Next' and 'Previous' buttons repeatedly, moving forwards and backwards through steps	Steps update correctly	Forward and backward navigation worked consistently across all steps, no errors or skipped steps	Pass
FR5	Theme selection	Switch from light to dark mode then navigate through application	Each UI page and widget uses new theme	User interface and all widgets switched to their dark mode colours	Pass
FR6	Text size selection	Increase text size then navigate through application	Text updates globally without breaking layout	All text in the system correctly updated to specified size and layout remained consistent	Pass
FR7	Glossary	Open glossary and click terms	Correct definition is displayed	Glossary page opened and all terms displayed correct definition when clicked	Pass
FR8	Progress tracking	Complete lesson then restart program	Progress remains saved and is visible	Progress was correctly saved and persisted after restarting, with lesson button still appearing green	Pass
FR9	Navigation	Move between all key sections	All pages load correctly and navigation is consistent	Navigation worked consistently across the system, back buttons worked to return to previous state. All navigation buttons did as was expected	Pass

These results demonstrate that all functional requirements were successfully met. Core features, such as animations, quizzes, navigation, and data persistence, worked as intended throughout testing. No major functional issues were identified during the course of testing, indicating that the system is able to function reliably.

6.3 Non-Functional Requirements Evaluation

ID	Requirement	How Evaluated	Evidence	Met?
NR1	System provides intuitive user interface	Usability heuristics and testing interactions	Consistent layout across screens, predictable navigation, clearly labelled navigation buttons	Yes
NR2	System supports users with varying visual needs	User interface and accessibility feature testing	Light/dark mode switching, adjustable text size, readable layout across all screens	Yes
NR3	System responds to input without noticeable delay	Observed during repeated testing	Immediate response to button press, smooth transitions between animation steps, no lag observed across system	Yes
NR4	System operates reliably without crashes	Observed during repeated testing	No crashes or loss of functionality observed during testing of all features	Yes
NR5	System runs on standard computer without internet	Testing system with and without internet access	System executed on windows 11 laptop using python environment, all features functional	Yes
NR6	System supports adding new lessons/features easily	Design evaluation	Modular codebase structure with separation of lesson files and reusable animation logic	Yes

This evaluation indicates that all non-functional requirements were met. The system demonstrates strong usability through consistent user interface design, as well as adherence to usability heuristics. Accessibility is also addressed through features such as adjustable text size and light/dark modes, allowing the interface to be customised to support user's needs. Performance and reliability were satisfactory, with the system responding immediately to user input, giving responsive feedback where appropriate, and no crashes observed during the testing process.

However, this evaluation is the result of developer testing and qualitative observations rather than formal user studies or extensive quantitative performance testing. Additionally, scalability of the system has been tested through the modular design of the system and the addition of a small number of lessons, rather than through implementation of a large number of lessons and extra features. As a result, whilst there is strong confidence that these requirements have been met, further validation could be achieved through system expansion and user testing.

6.4 Usability Evaluation

Usability of the system was assessed using usability principles defined by Nielson heuristics (Nielson, 1994). This evaluation of system usability was conducted through system testing and design evidence.

Num	Heuristic	Evidence	Met?
1	Visibility of system status	All buttons have obvious visual feedback when clicked and responses are immediate. Animation step indicator helps user understand where they are in a lesson	Yes
2	Match between system and real world	Simple language used for navigation. Buttons use as simplistic names as possible. Use of 'lesson' instead of more technical name so users can understand what it is immediately.	Yes
3	User control and freedom	All user interface pages (except main menu) have back buttons allowing user to navigate to previous interface state, avoiding the user getting stuck in a lesson for example.	Yes
4	Consistency and standards	Centralised design for simple user interface is common standard. User interface designs are consistent through using the same terminology across all pages and keeping a back button in the bottom left corner of every page.	Yes
5	Error prevention	Safeguards built into any forms on user interface, for example, custom GCM lesson's input menu has error checking and feedback and will not continue until the user has entered valid values. However, deleting progress is one button click which does not have verification to ensure it is not a mistake.	Partially
6	Recognition rather than recall	All options available to user at any given time are visible.	Yes
7	Flexibility and efficiency of use	Caters to novice users through simplistic design. The system does not include repetitive processes that would benefit from the addition of shortcuts.	Yes
8	Aesthetic and minimalist design	At any time while using the system, only necessary information is shown to the user. The design makes use of as few widgets as possible to keep interface minimal. Central design gives clear hierarchy to the design, with more important information being placed towards the middle of the page, and less important information being placed towards the edges or corners.	Yes

9	Help users recognise, diagnose and recover from errors	Error messages are given to the user when invalid values are entered into forms. Quiz feedback also includes message displaying to the user that the quiz has not been completed correctly.	Yes
10	Help and documentation	System design is self-explanatory, achieved through minimalist designs and following design standards (Nielson, 1994). Additional documentation is not necessary for this system.	Yes

This evaluation of the system against heuristics shows that the system scores highly for usability. The consistent and minimalist design allows users to quickly understand the system and engage with it. Navigation buttons are consistently placed on the user interface and behave as expected.

However, heuristic 5 is only partially met by the system. Whilst error checking and feedback is built into any forms the user may interact with, validation for irreversible actions such as deletion of progress is lacking.

The system could also improve usability through more advanced navigational options through lesson steps, such as allowing specific steps to be revisited without having to click 'previous' multiple times.

Despite these areas for improvement, the system broadly satisfies the usability principles set out by Nielson.

6.5 Success Criteria Evaluation

The following evaluates the extent to which the system meets the success criteria defined in Section 3.4.

ID	Criterion	Evidence	Achieved?
SC1	Feature functionality	Functional testing confirmed all features operate as defined, including animations, quizzes, glossary, progress tracking, settings and navigation (see Section 6.2)	Yes
SC2	Accuracy of lesson content	Lesson content was reviewed against established descriptions of algorithms, such as (Dworkin, 2001) and lecture content, with simplifications clearly explained within the lessons	Yes
SC3	Maintainability and extensibility	Modular structure, with separate lesson files and reusable animation logic allows new lessons to be added without modifying core components.	Yes

SC4	User interaction	Consistent layouts, uniform button behaviour, and predictable navigation was observed across all interfaces, supported by usability evaluation (see Section 6.4)	Yes
SC5	Learning effectiveness	Step-based animation design, combined with quiz and supporting glossary content, provide a structured learning experience that supports understanding of complex cryptographic concepts.	Yes

This evaluation indicates that the system meets the defined success criteria, and so can be considered successful. All core functionality operates as intended, supported by successful functional testing, and the system design allows for straightforward extension through its modular structure. User interaction is consistent and predictable, contributing to overall usability of the system.

However, learning effectiveness has been evaluated based on design and interaction rather than formal user studies. Current limited lesson support may also not be sufficient to fully validate maintainability and extensibility of the system. Despite this, the system demonstrates a strong level of success, achieving all its intended requirements and aims.

6.6 Further Testing and Evaluation

This testing methodology gives strong confidence that the system is successful, as all core features and requirements have been tested and verified, both during development and afterwards. In addition, no critical runtime errors were discovered during extended use of the system, and the application behaves consistently, despite different user flows. Meeting all functional, non-functional and success criteria requirements, as well as a strong usability evaluation, gives a strong level of confidence in the systems performance, usability and functionality.

However, despite these positive results, confidence is limited by the scope of the testing conducted. Testing has been performed solely by the developer, meaning it is likely to focus more on technical correctness rather than real-world user behaviour, and there is a risk that bias has interfered with observations to some extent. As a result, there is less certainty about the usability and accessibility of the system for first time users with no prior context of the application.

If more time were available, several areas of testing would be improved in order to increase confidence in results of these tests:

User testing and usability studies would be conducted with target users, students, as well as secondary users such as university staff. This would assess how effectively new users can navigate the system, understand the lesson content and engage with interactive elements such as quizzes. Feedback could be collected through questionnaires to identify areas of strength and areas to be improved.

Performance testing would be expanded through quantitative testing. Whilst current testing concluded that the system performs well under normal conditions, more rigorous testing could be conducted to measure performance under heavier workloads such as longer sessions and more interactions. Gathering quantitative data for metrics such as button response time and lesson load times would help to identify any areas of inefficiency and would also help to prove that requirements relating to performance and responsiveness have been met.

Accessibility testing would be an area of particular focus. The current design and accessibility features were informed by best practices and trustworthy sources that advise on how to improve accessibility. However, the effectiveness of the systems design has not been evaluated by those who benefit from these features.

Therefore, while the current implementation demonstrates a high level of reliability, performance, accessibility and usability, further structured testing with real users and defined metrics would strengthen confidence.

The testing approach chosen was primarily influenced by time constraints and the need to prioritise implementation of core features within the strict timeframe. As a result of this, testing and evaluation focused on ensuring functionality and system stability, rather than more time and organisationally intensive testing methods such as user studies or iterative evaluation throughout development. While this was effective in verifying that the system meets the requirements defined in Section 3, it limits the extent to which conclusions can be made about real world usability and accessibility. In addition developer testing introduces the possibility of bias, as the system is evaluated based on expected use cases rather than genuine interaction, which has the potential to alter results slightly.

Conclusion & Future Work

7.1 Conclusion

This project aimed to develop an interactive educational tool to support the understanding of cryptographic algorithms, specifically modes of operation, through visualisations. Cryptographic processes are often difficult for learners to grasp due to their complex nature and this system aimed to address this by presenting the algorithms in a structured way.

The resulting system implements a group of lessons focused on modes of operation, and the key algorithms and concepts that they rely on, including ones focused on GCM and CTR. These lessons make use of step-by-step animations to break down each algorithm into manageable stages. Lesson content is supported by quizzes, a glossary of key terms, and other features such as progress tracking and customisable interface settings. Together, these elements provide an intuitive learning tool that helps users build understanding of complex material.

Testing and evaluation of the system demonstrated that all core functional requirements were successfully met, with features present and operating as intended. Non-functional requirements were also satisfied, with the system showing strong usability, consistency, performance and reliability, despite limitations of the testing methodology. All success criteria were fulfilled, and usability evaluation using Nielson's heuristics scored highly, despite small areas for improvement.

One of the key outcomes of this project was the effectiveness of the animations in simplifying the complex cryptographic algorithms. By decomposing these processes into smaller stages and pairing them with explanatory text and interactive quizzes, the system reduces cognitive load and supports learning. However, this approach highlights the challenge of balancing simplification with accuracy, which required careful consideration to ensure that the accuracy of the explanations was not being sacrificed.

Despite these successes, the system has some limitations. The number of lessons that have been implemented is relatively small when compared to the wider topic of modes of operation, restricting the scope of the tool. The evaluation of learning effectiveness is based on design and usability heuristic analysis rather than formal user studies and feedback. As a result of this, whilst there is strong evidence from testing results that the system supports understanding of these concepts, it has not been empirically validated.

Overall, the project can be considered successful in achieving its aims. A functional and accessible educational tool has been delivered that demonstrates how interactive visualisation can be used to improve understanding of complex technical concepts.

7.2 Future Work

Based on this evaluation of the system, several areas for future development have been identified.

Content expansion is one area where the system could be significantly improved. In its current state, the system has lessons covering GCM and CTR modes of operation, however, other modes such as Electronic Code Book (ECB) and Cipher Block Chaining (CBC) exist, which could be added to achieve a greater coverage of the topic. Expanding the range of supported algorithms would improve the system's educational value and demonstrate its scalable design.

Usability evaluation demonstrated that the system performs well in this area, however areas for improvement were identified. Validation for irreversible actions, through 'are you sure?' prompts would help to decrease user error and frustration. Enhancing lesson navigation by allowing users to navigate to specific steps offers them greater levels of control, improving the user experience and supporting the learning experience.

Quiz functionality had to be reduced slightly from the intended design due to time constraints and implementation difficulties. With extra time, the usefulness of the quizzes could be expanded by adding more question types, difficulty levels and greater detail in feedback once submitted, which would give users a more accurate picture of their understanding and allow them to improve.

Due to technical limitations of both Python and Tkinter, animations had to be as simple as possible, which acted as a constraint on the design and storyboarding of lessons. Future iterations of the system could explore different frameworks and coding languages to find ones that have stronger animation support which could allow for more visually complex and interesting animations.

While accessibility was a core consideration during the design process, future work could increase the ability of the system to cater to a wider range of users. For example, an animation speed slider would support users with different learning requirements. Users who need more time to understand each step can slow down animations, while more confident users can increase speed to maintain engagement. Keyboard only navigation is another improvement that could be made, which would increase accessibility of the system to those with motor impairments.

Finally, a key area for future work would be reevaluating the system with real users. Conducting user studies would allow the effectiveness of the system to be measured more rigorously and give an opportunity for user feedback to inform future development plans. This would give a clearer picture of the educational impact of the system and a stronger direction for the future.

Reflection

This section reflects on the key decisions made throughout the development process of the project and considers how these influenced the outcome and the process. By reflecting on these decisions, areas for improvement for future projects can be identified.

The biggest learning from this project was the importance of planning and proper experimentation before committing to a design. Initially, Qt was chosen as the graphical user interface framework due to its more advanced options, with the belief that this would result in a higher quality product. However, this framework introduced unnecessary complexity and significantly slowed development, resulting in a transition to a simpler framework. Whilst this choice was the right one for the development of the project, increasing development speed significantly, it highlights the importance of proper research and experimentation before committing. Selecting tools based on the number of features led to wasted time and effort, which could have been saved by instead evaluating all possible options in depth before choosing one.

Evaluation of the system was performed solely through developer testing and heuristic analysis. This approach assumes usability based on conventions rather than proving it based on real user feedback and interaction with the system. This highlights the importance of planning time for user testing and feedback, as well as time for further improvement following this, which would allow the system to be evaluated to a much deeper level. This is especially important in a workplace environment, where any projects developed are going to be used, and so need to be provably useable and functional.

Time management was a skill developed throughout this project. The initial plan was created with little knowledge of how long certain features would take to implement, meaning that estimates were usually inaccurate, resulting in an ineffective plan. This meant that development of certain features, such as quizzes, had to be simplified in order to save time. This showed the value of fully evaluating the complexity of features before planning for their development.

Overall, this project demonstrated the importance of proper research, effective time management and more in-depth evaluation. In future projects, greater emphasis will be placed on the research phase, as well as testing afterwards. This will help to identify potential areas of inefficiency early and improve the overall effectiveness of the development process.

List of Abbreviations

Abbreviation	Meaning
XOR	Exclusive OR
CTR	Counter Mode
GCM	Galois/Counter Mode
ECB	Electronic Codebook Mode
CBC	Cipher Block Chaining
IV	Initialisation Vector

Glossary

XOR (Exclusive OR)

A binary operation that outputs 1 when two input bits differ and 0 when they are the same, commonly used in cryptography to combine data with a keystream.

Plaintext

The original, readable form of data before encryption.

Ciphertext

Encrypted form of data, which appears unreadable without the correct decryption process.

Encryption

The process of transforming plaintext into ciphertext using an algorithm and a key, protecting data from unauthorised access.

Decryption

The process of converting ciphertext back into plaintext.

Keystream

A sequence of pseudorandom values which is combined with plaintext (usually using XOR) to produce ciphertext.

Block cipher

A cryptographic algorithm that processes fixed-size blocks of data, transforming plaintext blocks into ciphertext blocks using a key.

Stream cipher

An encryption method that processes data one bit at a time by combining plaintext with a keystream.

Mode of operation

A method that defines how a block cipher is applied to data larger than a single block, often introducing new mechanisms to improve security.

Counter (CTR) mode

A mode of operation that converts a block cipher into a stream-like cipher

Galois/Counter mode

A mode of operation that combined CTR with an authentication mechanism, providing confidentiality and integrity of data.

Initialisation vector (IV) / Nonce

A value used alongside a key to ensure that encryption produces unique outputs, even with the same plaintext.

Authentication tag

A value generated in some algorithms that is used to verify the integrity and authenticity of the data.

References

- Stallings, W., Columbus, B., New, I., San, Y., Hoboken, F., Cape, A., Dubai, T., Madrid, L., Munich, M., Montréal, P., Delhi, T., São, M., Sydney, P., Kong, H., Singapore, S. and Tokyo, T. (n.d.). *CRYPTOGRAPHY AND NETWORK SECURITY PRINCIPLES AND PRACTICE SEVENTH EDITION GLOBAL EDITION*. [online] Available at: https://cdn.comparitech.com/wp-content/uploads/2025/03/cryptography-and-network-security_-principles-and-practice-7th-global-edition.pdf.
- Chernikova, O., Heitzmann, N., Stadler, M., Holzberger, D., Seidel, T. and Fischer, F. 2020. Simulation-based learning in higher education: A meta-analysis. *Review of Educational Research*, 90(4):499–541
- CrypTool Portal. (n.d.). *CrypTool Portal*. [online] Available at: <https://www.cryptool.org/en/ct2/>.
- Dworkin, M. (2001). *Recommendation for Block Cipher Modes of Operation Methods and Techniques*. [online] Available at: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>.
- Mack, R.L. and Jakob Nielsen (1994). *Usability inspection methods*. New York N.Y. Etc: Wiley & Sons
- Sweller, J. (1988), Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science*, 12: 257-285. https://doi.org/10.1207/s15516709cog1202_4
- W3C (2025). *Web Content Accessibility Guidelines (WCAG) 2.1*. [online] W3.org. Available at: <https://www.w3.org/TR/WCAG21/>.

Appendix

Appendix A: Testing Table

Functional requirements testing:

ID	R	Description	Expected Result	Actual Result	Outcome
1	FR1	Run animation for selected algorithm	Animation displays step-by-step progression	Animation progresses through steps correctly	Pass
2	FR1	Observe data changes during animation	Data updates correctly at each step	Data transformations are displayed accurately	Pass
3	FR2	Load GCM encryption lesson	GCM lesson loads and runs correctly	GCM lesson loads and functions as expected	Pass
4	FR2	Load CTR decryption lesson	CTR lesson loads and runs correctly	CTR lesson loads and functions as expected	Pass
5	FR3	Submit correct quiz answers	Correct feedback given and user can click finish	Feedback saying 3/3 is given and finish button appears	Pass
6	FR3	Submit incorrect quiz answers	Incorrect feedback given and user can retry	Feedback saying some answers incorrect given and retry button appears	Pass
7	FR3	Submit quiz without answering	Incorrect feedback given and user can retry	Feedback saying 0/3 given and retry button appears	Pass
8	FR4	Press 'Next' during animation	Moves to next step	Animation moves to next step	Pass
9	FR4	Press 'Previous' during animation	Moves to previous step	Animation returns to previous step	Pass
10	FR4	Press 'Next' at final step	Moves onto quiz	Quiz page appears	Pass
11	FR4	Press 'Previous' at first step	No errors, animation remains on first step	Previous button not clickable, animation stays on the same step	Pass
12	FR5	Switch to dark mode	Theme updates correctly	Interface updates to dark mode	Pass
13	FR5	Restart system after theme change	Theme persists	Selected theme is retained on restart	Pass
14	FR6	Increase text size	Text scales appropriately	Text size increases across UI	Pass

15	FR6	Decrease text size	Test scales appropriately	Text size decreases across UI	Pass
16	FR7	Select glossary term	Definition is displayed	Correct definition shown in display area	Pass
17	FR7	Switch between glossary terms	Definition is displayed	Previous definition is replaced by newly selected one	Pass
18	FR8	Complete lesson\quiz	Progress file is updated	Progress file updated to include entry about completed lesson	Pass
19	FR8	Restart application	Progress persists	Progress data persists after restart	Pass
20	FR8	View progress on lesson page	Progress shown accurately	Lesson buttons turn green to show completion	Pass
21	FR9	Navigate between sections	Navigation works correctly	All sections accessible without error	Pass
22	FR9	Use back buttons	Returns to correct screens	Back button works consistently across application	Pass

R = Requirement

Appendix B: Quiz question examples

GCM encryption lesson:

Q1 – How does GCM differ from CTR?

A1 – It doesn't

A2 – GCM encryption uses more rounds of encryption to increase security

A3 – GCM includes an authentication tag with the ciphertext

Answer = A3

Q2 – What type of multiplication is used when creating the authentication tag?

A1 – Galois Field Multiplication

A2 – Normal Multiplication

A3 – It doesn't use multiplication

Answer = A1

Q3 – What must never be reused in GCM mode?

A1 – The key

A2 – The ciphertext

A3 – The IV / Nonce

Answer = A3

CTR encryption lesson:

Q1 – What does CTR mode use to generate a keystream?

A1 – An incrementing counter encrypted with AES

A2 – The plaintext XOR with the key

A3 – A random number generator

Answer = A1

Q2 – What operation is used to combine the keystream with the plaintext?

A1 – AND

A2 – XOR

A3 – OR

Answer = A2

Q3 – What must never be reused in CTR mode?

A1 – The key

A2 – The ciphertext

A3 – The IV / Nonce

Answer = A3

XOR lesson:

Q1 – Which statement best describes XOR?

A1 – Outputs 1 when both inputs are the same

A2 – Outputs 1 when both inputs are different

A3 – Always outputs 0

Answer = A2

Q2 – Which one of these is a characteristic of XOR?

A1 – Reversible

A2 – Fast

A3 – Both

Answer – A3

Q3 – What is the result of 1 XOR 1?

A1 – 1

A2 – 0

A3 – It depends on the key

Answer = A2

Appendix C: Persistent Data Storage Examples

Glossary.txt excerpt:

CTR Encryption//A mode of operation which transforms block ciphers into stream ciphers. It works by incrementing a counter value (combined with a nonce) and encrypting that with the key to create keystreams. These keystreams are then XORed with the plaintext to produce ciphertext. For more information, see "CTR Encryption" lesson.**

GCM Encryption//A mode of operation based upon CTR that provides both encryption and authentication. It encrypts in the same way as CTR, but also generates an authentication tag. For more information, see "GCM Encryption" lesson.**

Nonce//A number used once in encryption to ensure the same plaintext does not produce the same ciphertext**

Key//A secret value used by an encryption algorithm to encrypt and decrypt data. Should be kept secret at all times**

XOR//A logical operation often used in encryption algorithms. Compares two bits and outputs 1 if they are different and 0 if they are the same**

Block Cipher//An encryption algorithm that encrypts data using fixed size blocks**

Stream Cipher//An encryption method that processes data one bit at a time**

Plaintext//Original readable message that sender wants to send**

Ciphertext//Encrypted output of an encryption algorithm produced from the plaintext as well as other values such as a key or nonce**

Progress.txt excerpt:

XOR

Security Problem

CTR Encryption

GCM Encryption

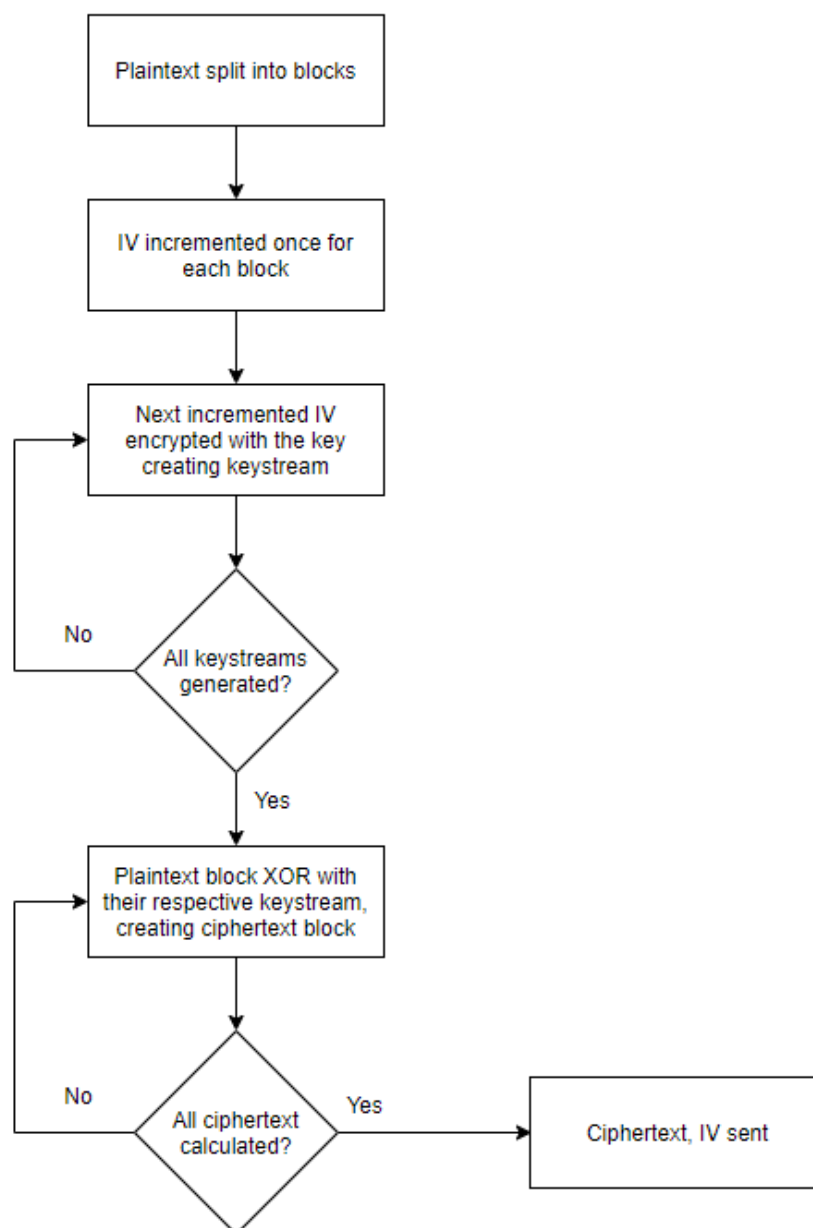
Settings.txt example:

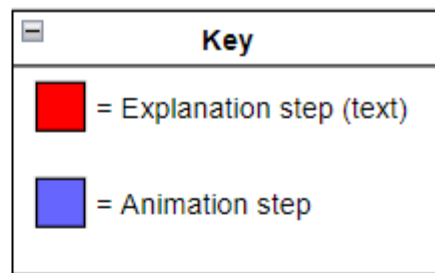
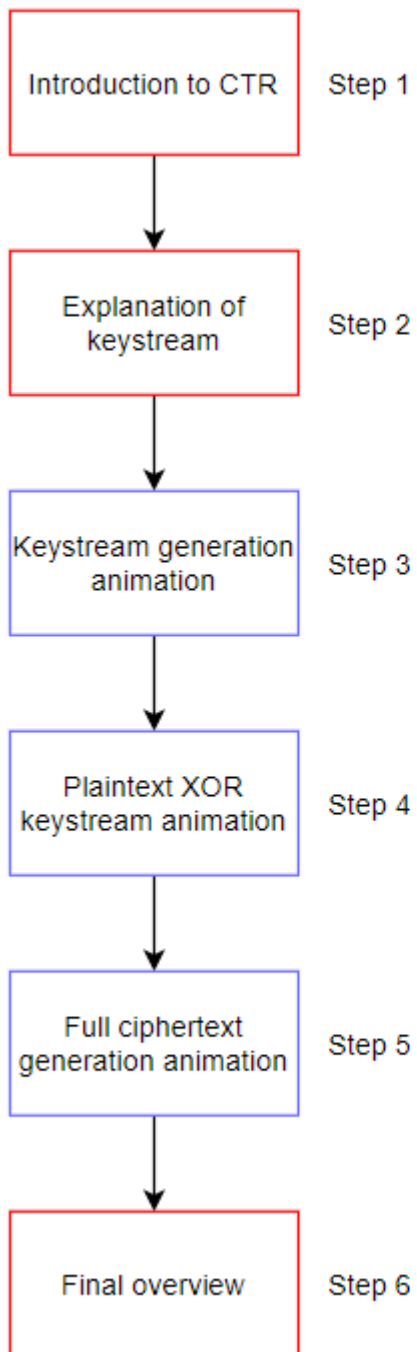
False

14

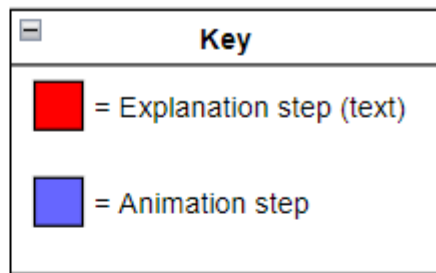
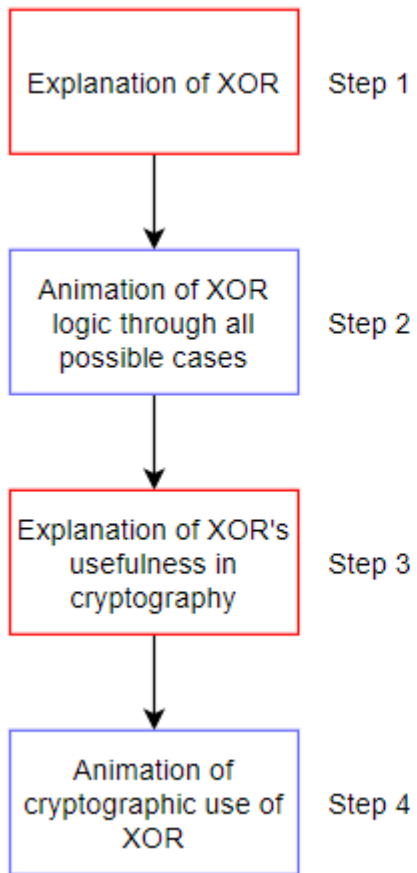
Appendix D: Lesson Design Diagrams

CTR lesson





XOR lesson:



Appendix E: Screenshots of lesson progression

0/6

CTR is a mode of operation that turns a block cipher into a stream cipher. It does this by creating keystreams to encrypt each block with. The main components of CTR are: Plaintext, Key, Initialisation Vector (Nonce). This nonce must never be reused.

For this example we will use the following values:

Plaintext: 0001,1100,0011

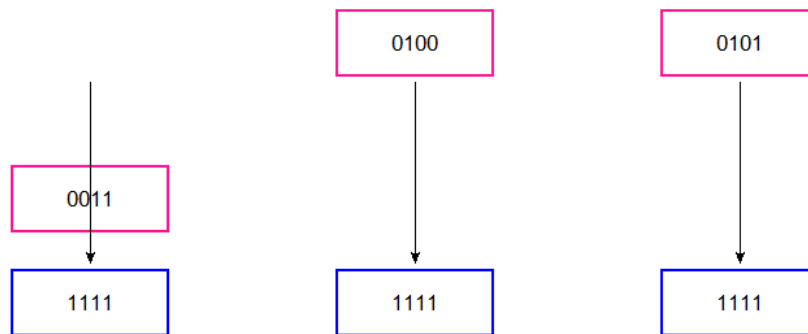
Key: 1111

IV: 0010

1/6

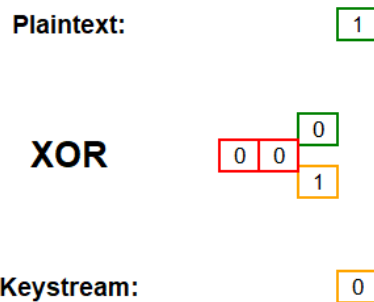
The first step is to create keystreams for each of the plaintext blocks
This is done by incrementing the IV and encrypting it with the key using AES
NOTE: For this demo AES not used due to size of the blocks,
simplified algorithm used instead to illustrate

Keystream Generation



Incremented IVs are encrypted with the key

Encryption of PT block 1



First plaintext block XORed with the first keystream

Encryption

Ciphertext Block 1

0011

Ciphertext Block 2

1111

Ciphertext Block 3

The other plaintext blocks are XORed with their respective keystream giving us the following full ciphertext

This ciphertext is then sent to the recipient alongside the IV
Using CTR is simple, however it is not as secure as it can be.

In future lessons, GCM will be explored, which is a mode of operation that builds upon CTR by adding authentication

Lesson complete! Press next to take the quiz

Quiz

Q1- What does CTR mode use to generate a keystream?

- ☒ An incrementing counter encrypted with AES
- ☐ The plaintext XORed with the key
- ☐ A random number generator

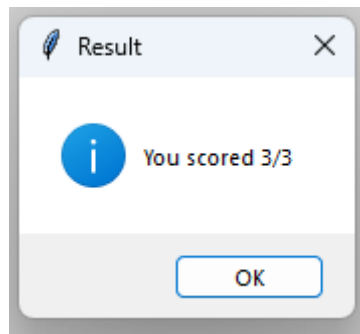
Q2- What operation is used to combine the keystream with the plaintext?

- ☐ AND
- ☒ XOR
- ☐ OR

Q3- What must never be reused in CTR mode?

- ☐ The key
- ☐ The ciphertext
- ☒ The IV / Nonce

Submit



Quiz

Q1- What does CTR mode use to generate a keystream?

- ☒ An incrementing counter encrypted with AES
- ☐ The plaintext XORed with the key
- ☐ A random number generator

Q2- What operation is used to combine the keystream with the plaintext?

- ☐ AND
- ☒ XOR
- ☐ OR

Q3- What must never be reused in CTR mode?

- ☐ The key
- ☐ The ciphertext
- ☒ The IV / Nonce

Submit

Finish

Choose a lesson

Security Problem

XOR

CTR Encryption

GCM Encryption

GCM Encryption (custom)

CTR Decryption

GCM Decryption